

Digital Forensics Basics: A Step-by-Step Guide for Beginners

An Open-Source Approach

Weifeng Xu, Ph.D.; Lin Deng, Ph.D.; and Mohit
Ajaykumar Dhabuwala

© 2026 *Dr. Xu's Research Lab*, Baltimore, Maryland
ALL RIGHTS RESERVED.

2026, Second Edition
E-book created September 7, 2026

Preface

Digital evidence now shapes criminal investigations, incident response, corporate inquiries, and everyday decisions about security and privacy. Yet beginning students often encounter digital forensics as a collection of unfamiliar tools, commands, and screenshots. This book was written to provide a more coherent starting point: an accessible, open-source, hands-on introduction that connects each technical action to an investigative question and to the evidence needed to answer it.

The book begins with one foundation chapter and continues with thirteen instructional chapters. Together, they move from computer-system context, representation, and Linux skills to evidence acquisition, file-system analysis, evidence searching, data carving, network traffic analysis, and concealed data in images. The goal is not to make readers memorize a single tool or procedure. It is to help them develop habits that remain useful as tools change: preserve original evidence, work from verified copies, record what was done, interpret output cautiously, and distinguish an observation from the conclusion it may support.

The chapters are organized around connected laboratory scenarios and evidence questions rather than isolated software demonstrations. A USB image, a deleted file, a network capture, or a suspicious image becomes meaningful only when it is examined in context and linked to a defensible explanation. This organization lets readers see how foundational skills support later work: acquisition enables analysis, analysis creates leads, and multiple artifacts can corroborate or challenge an interpretation.

This text is intended for undergraduate and graduate students, instructors, self-directed learners, and early-career practitioners who want a practical entry point into digital forensics. It is suitable for introductory digital-forensics courses, cybersecurity and criminal-justice technology programs, and professional training that needs guided laboratory activities. Readers do not need deep prior forensic experience, but they should be prepared to work carefully, ask questions about

evidence, and practice in authorized environments.

The official book companion site at <https://github.com/frankwxu/Digital-Forensics-Basic-Book> provides chapter-aligned presentations, updates, and related supporting information. We encourage instructors to adapt the labs to their course objectives and learners to repeat them with small, documented variations. When an activity requires a training file, the relevant chapter provides the instructions and link needed to obtain it.

For practical guidance on navigating chapters, reading commands and screenshots, using the companion materials, and studying effectively, begin with Chapter , *How to Use This Book*. Chapter 1 then establishes the disciplinary foundation of digital forensics and its relationship to cybersecurity. We hope this sequence helps readers approach each lab not as a script to copy, but as an opportunity to practice careful, reproducible forensic reasoning.

Weifeng Xu, Ph.D.

Lin Deng, Ph.D.

Mohit Ajaykumar Dhabuwala

Baltimore, Maryland

2026

Contents

Preface	ii
How to Use This Book	xiv
1 Who This Book Is For	xiv
2 How the Chapters Work	xiv
3 Companion Materials	xv
4 Reading the Commands, Screenshots, and Notes	xvi
5 How to Get the Most from the Labs	xvi
1 Introduction to Digital Forensics and Its Relationship to Cybersecurity	1
1.1 Introduction to digital forensics	2
1.2 Why digital forensics matters	2
1.3 Core goals of a digital forensic investigation	3
1.4 Digital forensics and cybersecurity	3
1.5 How digital forensics supports incident response	4
1.6 Types of digital evidence	4
1.7 The digital forensic investigation process	5
1.8 Evidence integrity and chain of custody	6
1.9 Legal, ethical, and privacy considerations	6
1.10 Digital forensics careers and roles	6
1.11 Tools and laboratory environments	7
1.12 How this book is organized	7
1.13 Summary	8
2 Number Systems for Digital Forensics	10
2.1 Number system conversions in digital forensics	11
2.1.1 Linux-Based Number System Conversions	11
2.1.2 Python-Based Number System Conversions	16
2.2 ASCII: Bridging human-readable text and binary code	17
2.2.1 Understanding Character Encoding: ASCII vs. UTF-8	18

2.2.2	Linux ASCII Conversions	20
2.3	ASCII Conversions in Python 3	21
2.4	Understanding UNIX time: A numerical representation of time .	22
2.5	UNIX Time (Python version)	24
2.5.1	Getting Current Time and Timestamps	24
2.5.2	Converting Timestamps to Readable Time	26
2.5.3	Comparing Epochs and Time Differences	28
2.6	Understanding hash codes in investigations	29
2.6.1	Command-Line Hashing	29
2.6.2	Python Hashing	30
2.7	The significance of byte order in digital forensics	31
2.7.1	Byte order identification	31
2.7.2	Contrasting byte ordering schemes: Little Endian vs. Big Endian	32
2.8	Summary	33
3	Computer Systems and Digital Evidence	35
3.1	Computer systems as evidence environments	35
3.2	Core components and their forensic relevance	37
3.3	Runtime system context and evidence correlation	40
3.4	Hard disks as forensic evidence sources	43
3.5	Partitions, volumes, and drive letters	46
3.6	Bootng process and partition-table overview	50
3.7	Master Boot Record	52
3.8	GUID Partition Table	59
3.9	Summary	65
4	Examining Digital Evidence with Linux	67
4.1	Introduction to Operating System (OS)	68
4.2	Introduction to File System	70
4.3	The Virtual File System: A universal translator for file systems .	72
4.4	Navigating the Linux command line	75
4.4.1	Getting started in the terminal	75
4.4.2	Exploring files and directories from the terminal	80
4.5	Understanding inodes in Linux	85
4.5.1	Inspecting inode and filesystem metadata	85

4.6	Operations on files and directories	90
4.6.1	Creating and inspecting files and directories	90
4.6.2	Copying, moving, and deleting files	93
4.7	Understanding and managing the PATH variable	97
4.7.1	Working with file paths and command lookup	98
4.8	Finding specific text within files	102
4.8.1	Using grep to search text and files	102
4.9	Investigating network activity in Linux	106
4.9.1	Testing connectivity and inspecting ports	106
4.9.2	Downloading and examining retrieved files	111
4.10	Creating shell scripts	114
4.10.1	Writing and inspecting a simple shell script	115
4.11	Modifying file permissions with the chmod command	117
4.12	Summary	120
5	Automating and Scaling Forensic Examination with Linux	123
5.1	Input and Output Devices as Files	123
5.1.1	Understanding the default input stream: stdin	125
5.1.2	Understanding the default output stream: stdout	126
5.1.3	Handling and redirecting the error messages	129
5.2	Communication and Data Transfer in Linux for Digital Forensics	130
5.3	Making cross-platform data transfers	134
5.3.1	Transferring files from Kali Linux to Windows 10	134
5.3.2	Create a backdoor on Windows 10 / remote file acquisition	138
5.4	Reverse Shells: A Forensic Perspective	141
5.5	Bypassing firewalls with outbound connections	142
5.6	Remote disk acquisition in digital forensics	142
5.6.1	Key considerations for remote forensic imaging	143
5.6.2	Assessing Disk Space Availability for Acquisition	144
5.6.3	Remote Disk Acquisition: Linux - receiving the image	146
5.6.4	Acquiring Windows disk partition remotely	147
5.6.5	Verifying the file transfer	150
5.7	Addressing slow remote disk acquisition	151
5.7.1	Automating the Experimentation (Linux Only)	153
5.7.2	Manual Block Size Testing	155
5.8	Summary	156

- 6 Digital Forensics Fundamentals: USB Image Analysis with Autopsy 158**
 - 6.1 What is digital forensics 158
 - 6.1.1 Steps of digital investigation 159
 - 6.1.2 Evidence collection and handling 159
 - 6.1.3 Acquiring a Forensic Image of Evidence Media 160
 - 6.2 Investigating a USB Disk Image with Autopsy 161
 - 6.2.1 Introduction to Autopsy 161
 - 6.2.2 Preparing the Autopsy environment and evidence image . 161
 - 6.3 Analyzing a USB image with Autopsy 165
 - 6.3.1 Creating a New Case 165
 - 6.3.2 Adding the USB image as a data source 168
 - 6.4 Recovering the deleted content from the USB image 170
 - 6.4.1 Finding the files deleted from the USB 170
 - 6.4.2 Tagging deleted files for review 171
 - 6.4.3 Recovering and extracting the tagged files 173
 - 6.5 Generating the final forensic report 176
 - 6.6 Summary 179
- 7 File-System Evidence Analysis with The Sleuth Kit 181**
 - 7.1 Analyzing the forensic image: Autopsy and The Sleuth Kit . . . 182
 - 7.2 File system and its layers 186
 - 7.3 The Host Protected Area (HPA) 188
 - 7.4 Examining a USB drive using TSK 190
 - 7.4.1 Preparing the USB image for analysis 190
 - 7.4.2 Identifying the image and file system 193
 - 7.4.3 Listing and recovering deleted files 195
 - 7.4.4 Examining raw blocks and hexadecimal content 198
 - 7.5 Accessing USB Image Content 201
 - 7.6 Summary 204
- 8 Acquiring and Verifying USB Evidence 206**
 - 8.1 Understanding disk images 207
 - 8.1.1 File formats for disk images 207
 - 8.1.2 Image acquisition 207
 - 8.2 USB image acquisition in Windows 208
 - 8.2.1 Acquiring a USB image with FTK Imager 208

8.3	Setting up a Kali Linux virtual machine on a Windows host . . .	214
8.3.1	Preparing the Kali virtual machine	215
8.4	Imaging a USB drive in Linux	218
8.5	Image acquisition over the network	220
8.5.1	Kali as sender, Windows as receiver	220
8.5.2	Windows as sender, Kali as receiver	223
8.6	Summary	225

9	Digital Evidence Search: The Power of Regular Expressions in Digital Forensics	228
9.1	What is Regular Expression (regex)?	229
9.1.1	Understanding the Grep command and its role in pattern matching	229
9.1.2	The regex cheatsheet	230
9.2	Hands-on regex lab	230
9.2.1	Preparing the lab files	232
9.2.2	First searches in the roster files	234
9.3	Word character, word string, and word boundary	237
9.3.1	Two-character words and word boundaries	238
9.3.2	Whole-word matching on realistic data	241
9.4	Conditional matching in regex: Lookaround assertions	245
9.4.1	Lookbehind assertions	246
9.4.2	Lookahead assertions	247
9.4.3	Mismatch in lookups	249
9.4.4	Getting exact matches	249
9.5	Matching phone numbers	251
9.5.1	Matching any 10-digit number	251
9.5.2	Matching exact 10-digit number	252
9.5.3	Matching phone numbers with period separators	253
9.5.4	Matching phone numbers with or without period separators	254
9.5.5	Matching phone numbers with multiple separators	254
9.5.6	Matching Phone Numbers with Multiple Formats	256
9.6	Matching email addresses and passwords	257
9.6.1	Matching a basic email pattern	258
9.6.2	Matching email addresses in the text file	259
9.6.3	Check if a password is valid	260

9.7	Applying Regex on Java source code	261
9.7.1	Search string in Java and show line numbers	261
9.7.2	Counting keyword occurrences in Java source code	262
9.8	Extracting artifacts from HTML, XML, and document formats	264
9.8.1	Extracting HTML tags and content with backreferences	265
9.8.2	Matching HTML tags using Lazy quantifier	266
9.8.3	Extracting Content within HTML Tags	267
9.9	Matching network and low-level artifacts	269
9.9.1	Matching IPv4 addresses with groups	269
9.9.2	Matching HTTP requests and URLs	270
9.9.3	Extracting hexadecimal numbers	273
9.9.4	Matching MAC addresses in text	274
9.9.5	Matching MAC addresses from a PCAP file	276
9.9.6	Analyzing XML inside .docx files	280
9.9.7	Using sed for document cleanup and extraction	284
9.10	Searching disk images with find and metadata	295
9.10.1	Understanding the find command for evidence search	296
9.10.2	Preparing and examining the evidence image	296
9.10.3	Advanced file search techniques with the find command	301
9.10.4	Pattern-based filename searches with find	309
9.10.5	Filtering file paths with find's -regex option	312
9.11	Filtering files by permissions, timestamps, size, and ownership	317
9.11.1	Identifying files with full permissions	318
9.11.2	Identifying files without full permissions	319
9.11.3	Locating files with specific permissions	320
9.11.4	Understanding file timestamps: access time, modify time, and change time	322
9.11.5	Using find to identify files based on timestamps	331
9.11.6	Finding files accessed within a specific timeframe	334
9.11.7	Using find to identify files based on size	335
9.11.8	Locating files belonging to a specific user	340
9.12	Anti-Forensics: The challenge of timestamp manipulation	341
9.12.1	Setting fake timestamps by changing system time	342
9.12.2	Restoring system time to the correct value	343
9.12.3	Setting a fake time using touch -t	344
9.12.4	Copying files and preserving timestamps	347
9.12.5	Examining command history for forensic clues	348

9.13 Summary	350
10 Data Carving: Recovering Lost and Hidden Data	352
10.1 Accessing data units (Blocks) of a file in Linux EXT4 file system	353
10.1.1 Downloading the .JPG image file	353
10.1.2 Examining file metadata and block allocation	355
10.2 Getting file information and signature of a .jpg file	356
10.2.1 Examining the internal structure of JPEG file	358
10.2.2 Image compression technique: Huffman Coding	359
10.3 Exploring the xxd Utility for Forensic Analysis	361
10.3.1 Understanding the file content structure	362
10.3.2 Examining JPEG File Signatures with xxd	363
10.3.3 Alternative method of viewing JPG header and tail signatures	364
10.3.4 Viewing specific portions of a file	365
10.4 Utilizing hexadecimal analysis for file type identification	366
10.4.1 Generating plain hex dumps	366
10.4.2 Searching for signatures in plain hex dumps	367
10.5 Introduction to the tr (translate) command	368
10.5.1 Implementing tr command for text manipulation	368
10.5.2 Removing newlines (\n) from hex dumps	369
10.5.3 Restructuring text with tr	370
10.5.4 Matching content between header and tail signatures . . .	374
10.5.5 Reversing a hex dump to its original binary form	375
10.6 Interesting evidence in file header	376
10.6.1 Finding the author information	376
10.6.2 Viewing some specific metadata	378
10.6.3 Viewing author name and offset	379
10.7 Fake the date (2020 - 2030)	379
10.8 Extracting images from a word document	381
10.8.1 Preparing and downloading the lab files	382
10.8.2 Installing Bless Hex Editor with Flatpak	383
10.9 Analyzing the hex dump in bless hex editor	386
10.9.1 Opening the document in Bless	386
10.9.2 Locating the embedded JPEG boundaries	387
10.9.3 Extracting, saving, and viewing the carved image	389
10.10 Using alternative approaches for file carving	394
10.10.1 File carving with the xxd command	394

10.10.2	File carving with the binwalk command	396
10.10.3	File carving with the binwalk -e command	398
10.11	Carving/Recovering a USB image	399
10.11.1	Prepare a USB image for file carving	399
10.11.2	Viewing the content of the zipped file	401
10.11.3	Extracting the contents of the 7z archive	402
10.11.4	Verifying the integrity of the extracted USB image file . .	403
10.12	Examining the extracted USB content	404
10.12.1	Identifying partition structure	405
10.12.2	Selecting file types for carving	406
10.12.3	Identifying and listing deleted files on the USB image . .	407
10.13	Introduction to scalpel command	410
10.13.1	Accessing the Scalpel help page	410
10.13.2	Carving files from the USB image	411
10.13.3	Viewing the carved output as a directory tree	413
10.13.4	Reviewing the Scalpel audit log	414
10.13.5	Interpreting the carved images	416
10.14	Summary	418

11	Wireshark for Digital Investigations - Part 1: Capturing, Filtering, and Examining HTTP and TCP Communications	420
11.1	Understanding Wireshark's command-line options	421
11.2	Capturing and analyzing web traffic with Wireshark	422
11.2.1	Creating a simple HTML web page	422
11.2.2	Starting Apache and testing the HTML page	423
11.2.3	Modifying the HTML file permission	424
11.2.4	Retrieving HTML file information with the curl command	425
11.3	Capture HTTP traffic using Wireshark	427
11.3.1	Preparing Wireshark for non-root capture	427
11.3.2	Preparing the browser and capture files	431
11.3.3	Opening a saved capture or starting live capture	432
11.4	Establishing TCP connections: The three-Way handshake	435
11.4.1	Capturing the SYN packet: Initiating the TCP handshake	436
11.4.2	Capturing the SYN-ACK packet: Server's acknowledgment and response	437
11.4.3	Capturing the ACK packet: Finalizing the TCP handshake	439

11.5	Analyzing HTTP requests	440
11.5.1	Filtering and analyzing HTTP Requests	441
11.5.2	TCP segment and payload analysis of an HTTP request	442
11.5.3	Analyzing IP and MAC address details within the HTTP request	444
11.5.4	Acknowledgment (ACK) packet in the HTTP request	446
11.6	Examining the HTTP response	448
11.6.1	Dissecting the captured response	448
11.6.2	Examining the HTTP response data	449
11.6.3	Analyzing the TCP segment of the HTTP response data	450
11.6.4	Analyzing the client's TCP acknowledgment	452
11.7	Ending the TCP connection	453
11.7.1	Client-initiated TCP connection termination	454
11.7.2	Server acknowledgment and connection closure	455
11.8	Summary	456
12	Wireshark for Digital Investigation - Part 2: Image Extraction	458
12.1	Capturing and examining HTTP traffic with embedded image	459
12.1.1	Using the provided capture	459
12.1.2	Acquiring the Wireshark capture log file	459
12.2	Analyzing the HTTP GET request and response	460
12.2.1	Analyzing the HTTP GET image request and response	461
12.2.2	Calculating the length of the TCP segments	462
12.2.3	Dissecting the raw data of TCP Payload	464
12.2.4	Determining the HTTP response header length	465
12.2.5	Identifying image boundaries within TCP segments	466
12.3	Extracting image from the HTTP GET request	468
12.3.1	Exporting the packet byte containing the .JPG file	468
12.3.2	Saving the exported packet as .jpg file	470
12.3.3	Visualizing the extracted image	471
12.3.4	Documenting the recovered image	472
12.4	Relationship between IP packet and TCP payload	472
12.4.1	Dissecting the structure of TCP (Transmission Control Protocol) header	473
12.4.2	Understanding the IP (Internet Protocol) header structure	474
12.4.3	Examining the relationship between IP Packet and TCP payload size	476

12.5 Summary	478
13 Detecting and Analyzing Concealed Data in Digital Images	480
13.1 Steganography Techniques: Substitution	481
13.2 Hands-on with steganography techniques	485
13.2.1 Acquiring the .bmp Image	485
13.2.2 Viewing the downloaded image	486
13.2.3 Viewing the Steghide help page	487
13.2.4 Creating the secret message text file	488
13.3 Embedding data within an image using Steghide	489
13.3.1 Embedding a text file within an image	490
13.3.2 Extracting embedded data with Steghide	491
13.4 Cracking the password using a wordlist	492
13.5 Summary	494
14 Conclusion and Next Steps in Digital Forensics	496
14.1 Bringing the Skills Together	497
14.2 What a Careful Examiner Must Remember	497
14.3 Ethics, Accuracy, and Professional Responsibility	498
14.4 Using the Companion Materials to Keep Learning	498
14.5 Where to Go Next	499
14.6 Summary	499
The Author: Weifeng Xu, Ph.D.	501
The Author: Lin Deng, Ph.D.	502
Glossary	503
Bibliography	510

How to Use This Book

This opening guide explains how to navigate the book and work effectively with its labs. The chapters that follow are practical, tool-driven, and evidence-focused, using command-line workflows, screenshots, sample artifacts, and short investigative exercises. Read this guide first, then begin Chapter 1 for the conceptual foundation of digital forensics and its relationship to cybersecurity.

1 Who This Book Is For

This book is intended for students, instructors, and self-directed learners who want a practical introduction to digital forensics. It is especially useful for readers who learn best by doing: running commands, examining files, comparing outputs, and connecting technical observations to investigative meaning.

The book assumes curiosity and persistence more than deep prior expertise. Some chapters begin with foundational computer skills, while later chapters move into image analysis, metadata inspection, network traffic review, data carving, and steganography. If a chapter feels advanced at first, use the earlier chapters and the companion materials as a bridge rather than treating each topic as isolated.

2 How the Chapters Work

Most chapters follow a recurring pattern:

Learning Objectives Each chapter opens with measurable goals that identify the concepts, evidence types, and practical skills you should be able to explain or apply.

Orientation A short conceptual opening explains what the tool, artifact, or forensic method is and why it matters.

Hands-on workflow The chapter then shifts into practical steps using commands, screenshots, and lab files.

Interpretation Important outputs are connected back to a forensic question, such as what was accessed, when an action occurred, what data was hidden, or how a file can be recovered.

Summary Each chapter closes by reinforcing the main investigative lessons and the role of the technique in real casework.

Chapter Review Review questions, discussion prompts, and a hands-on exercise help you check understanding, reason through investigative tradeoffs, and apply the chapter's methods.

Readers will get the most value from the book by treating each chapter as an investigation rather than a script to copy mechanically. When you see a command, pause long enough to ask what evidence it is meant to reveal, what part of the output matters, and how that output supports a forensic conclusion.

3 Companion Materials

The official book companion site is <https://github.com/frankwxu/Digital-Forensics-Basic-Book>. It provides chapter-aligned presentations, updates, and related supporting information.

The shared, scenario-based approach used by these materials is described in Xu et al. [2022].

When a chapter asks you to download a file, inspect a sample artifact, or compare your results to a screenshot, follow the instructions and link provided in that chapter. Use the companion site to locate the matching presentation and other reader resources. Using these materials with the book will make the learning process smoother and will help you reproduce the examples more reliably.

4 Reading the Commands, Screenshots, and Notes

This book uses a consistent hands-on style. To follow it more easily, keep these conventions in mind:

Commands Commands shown in bold or monospaced form are meant to be typed exactly unless the text tells you to adapt a filename, path, or interface name.

Screenshots Screenshots are not decorative. They show what you should expect to see after a step succeeds and often highlight the part of the output that matters most.

Artifacts Filenames, hashes, packet captures, image files, and directories are evidence-bearing items. Pay attention to their names and locations because later commands often build on earlier artifacts.

Interpretation A command by itself is not the goal. The goal is to understand what the result means in an investigative context.

Warnings and notes Short notes are included to help you avoid common mistakes, recognize environment differences, and distinguish normal behavior from suspicious evidence.

Many chapters assume a Linux-based lab workflow, often with Kali Linux or another forensic workstation environment. If your local paths, permissions, browser cache locations, or installed tools differ from the examples, focus first on the investigative purpose of the step and then adapt the command as needed for your environment.

5 How to Get the Most from the Labs

The most effective way to use this book is to work actively. Type the commands, inspect the outputs, and compare what you see to the figures. If a result differs from the screenshot, do not assume failure immediately; instead, ask whether the difference comes from timestamps, permissions, software versions, network state, or changes in the sample data.

It is also helpful to keep brief notes while you work. Record the commands you ran, what evidence they exposed, and what conclusion each step supported. That

habit will make the later reporting chapters more meaningful and will strengthen your ability to explain forensic reasoning clearly.

Use the exercises only in the authorized lab environment and with the provided training materials. Record your process carefully, preserve your notes with the lab results, and adapt examples only within the authority and scope granted for your work.

1 Introduction to Digital Forensics and Its Relationship to Cybersecurity

The guide *How to Use This Book* explains how to navigate the labs, read commands and screenshots, and use the companion materials. This chapter has a different purpose: it establishes the concepts and evidence practices that give those activities forensic meaning.

Digital devices record traces of human activity, system behavior, and security events. Digital forensics is the disciplined process of identifying, preserving, examining, analyzing, and reporting those traces so that conclusions can be explained and, when necessary, defended. The evidence may come from a computer, mobile device, removable drive, cloud service, network capture, or application log.

This book develops practical investigation skills, but technical skill alone is not enough. An analyst must also protect the original evidence, document each action, distinguish observations from conclusions, and recognize the limits of the available data. This chapter establishes that foundation and shows how forensic work connects to cybersecurity.

Learning Objectives

- Define digital forensics and distinguish it from cybersecurity and incident response.
- Describe the identification, preservation, examination, analysis, and reporting stages of an investigation.
- Explain why evidence integrity, chain of custody, scope, and authorization matter.

- Classify common sources of digital evidence and identify their limitations.
- Relate the later chapters of this book to a complete forensic workflow.

1.1 Introduction to digital forensics

Digital forensics applies scientific and repeatable methods to digital evidence. An investigation may seek to determine what happened, when it happened, which accounts or systems were involved, and what data was affected. The result is not merely a collection of files or screenshots; it is a reasoned account that another qualified examiner can review.

Digital evidence may arise from user activity, applications, operating systems, and hardware; Chapter 3 examines those layers as evidence environments.

Digital forensic work appears in criminal investigations, civil disputes, internal corporate inquiries, incident response, regulatory matters, and recovery from operational failures. The same artifact can answer different questions depending on the case. A USB drive, for example, might reveal evidence of file transfer, malware delivery, intellectual-property theft, or ordinary authorized work.

1.2 Why digital forensics matters

Digital evidence is often fragile. A system can overwrite logs, rotate temporary files, synchronize cloud data, or change timestamps during normal use. An investigator therefore needs to make sound preservation decisions before beginning detailed analysis.

Forensics also gives technical findings context. A security alert may show that suspicious activity occurred, while a forensic examination can correlate logs, files, network traffic, and metadata to explain the sequence of events and the scope of the impact.

A running example. Imagine that an organization detects unusual network traffic from an employee workstation shortly after a USB drive is connected. The security team must contain the potential threat, but the forensic questions are broader: Which USB device was used? What files were copied or executed?

What network connections followed? Did the activity affect other systems? The techniques in this book help answer those questions while preserving a record of how each conclusion was reached.

1.3 Core goals of a digital forensic investigation

The central goals of an investigation are to locate relevant evidence, preserve it without unnecessary alteration, interpret it accurately, and communicate conclusions clearly. A strong examination separates what is directly observed from what is inferred.

For example, a browser-history record may directly show that a URL was stored in a particular user profile. It may support an inference that someone using that profile accessed the site, but it does not by itself prove the identity or intent of the person at the keyboard. This distinction is essential when reporting results.

1.4 Digital forensics and cybersecurity

Cybersecurity and digital forensics are closely related but have different primary purposes. Cybersecurity aims to reduce risk by protecting systems, detecting threats, and responding to suspicious activity. Digital forensics investigates digital traces to establish reliable facts after an event, or when an event is suspected.

The two disciplines reinforce each other. Good security controls create useful logs, alerts, backups, and asset records. Good forensic practices help security teams understand an intrusion, validate an alert, contain affected systems, and improve future defenses. In practice, an analyst may move repeatedly between security monitoring and forensic examination during the same incident.

Discipline	Primary purpose	Typical question
Cybersecurity	Reduce risk by protecting systems and detecting threats.	How can we prevent, detect, or limit this threat?
Incident response	Contain an active incident and restore operations.	What must we do now to reduce harm?
Digital forensics	Establish reliable facts from digital evidence.	What happened, how, when, and what evidence supports that conclusion?

1.5 How digital forensics supports incident response

Incident response focuses on quickly limiting harm and restoring normal operations. Digital forensics adds evidentiary discipline to that work. It can help determine the initial access path, affected hosts, attacker actions, data exposure, and the persistence mechanisms that must be removed [Kent et al., 2006, Nelson et al., 2025].

Speed matters during an incident, but so does careful documentation. Before collecting data, responders should record the system state, the collection time and timezone, the person performing the action, and the tools used. When possible, they should preserve original data and analyze verified copies.

1.6 Types of digital evidence

Digital evidence includes both the contents of data and the metadata surrounding it. Common sources include filesystems, operating-system logs, browser artifacts, email, memory, network captures, mobile-device records, cloud audit logs, and application databases.

No single source tells the whole story. File timestamps may be ambiguous, logs may be incomplete, and user-controlled data can be misleading. Investigators build confidence by correlating independent artifacts and documenting both sup-

porting and contradictory evidence.

For example, a data-leakage investigation may require evidence from a device, its operating system, and supporting network infrastructure before the sequence of events can be reconstructed [Xu et al., 2019].

1.7 The digital forensic investigation process

Although organizations use different procedures, a typical forensic process has five connected stages [Kent et al., 2006]:

1. **Identification:** Define the question, scope, potential evidence sources, and legal or organizational authority.
2. **Preservation:** Protect original evidence from loss or alteration and record its condition and custody.
3. **Collection and examination:** Acquire data using appropriate methods, verify copies, and extract relevant artifacts.
4. **Analysis:** Correlate artifacts, test explanations, establish timelines, and assess limitations.
5. **Reporting:** Present methods, findings, supporting evidence, and conclusions in language suited to the intended audience.

These stages are iterative. A finding during analysis may require additional collection, a narrower scope, or a revised hypothesis. Maintaining clear notes makes those decisions reviewable.

Applying the process. In the USB scenario, identification begins by defining the suspected workstation, USB device, and relevant time period. Preservation includes recording the device condition and creating a verified forensic image before examination. The examiner can then analyze the image for file-system artifacts while correlating those findings with workstation logs and the network capture. The final report should distinguish confirmed observations—for example, that a particular file existed on the USB image—from inferences about how or why it was used.

Collection choices also depend on the system state. A powered-off USB drive can usually be acquired as a storage image. A running computer may contain volatile evidence, such as memory contents and active network connections, that can disappear when it is shut down. Follow the authorized procedure for the case,

record the system state, and balance the need to preserve volatile data against the risk of altering evidence.

1.8 Evidence integrity and chain of custody

Evidence integrity means being able to show that the data examined is the same data that was collected, or that any changes are known and explained. Cryptographic hashes are commonly used to verify that a forensic image or exported file has not changed between collection and analysis [Kent et al., 2006, Scientific Working Group on Digital Evidence, 2023].

Chain of custody is the documented history of evidence handling. It should record who collected the item, when and where it was collected, how it was stored, who received it, and each transfer or examination. The goal is accountability: a reviewer should be able to trace the evidence from its source to the final report.

1.9 Legal, ethical, and privacy considerations

Investigators must work within applicable laws, organizational policy, and the authority granted for the examination. Requirements for consent, warrants, retention, monitoring, and disclosure vary by jurisdiction and organization. Scope matters: access to one system or account does not automatically authorize examination of every connected device, personal file, or cloud service.

Ethical practice also requires restraint. Collect only what is necessary, protect sensitive information, avoid unsupported claims, and report limitations candidly. When legal requirements or organizational procedures are uncertain, analysts should seek appropriate guidance before expanding collection or disclosure.

1.10 Digital forensics careers and roles

Digital forensic responsibilities appear in law enforcement, corporate security, consulting, legal support, government, and research. Common roles include forensic examiner, incident responder, malware analyst, e-discovery specialist, threat

hunter, and security operations analyst.

Titles vary, but successful practitioners combine technical curiosity with careful documentation and clear communication. They must explain complex artifacts to audiences that may include managers, attorneys, incident commanders, or a court.

1.11 Tools and laboratory environments

Forensic tools help acquire data, calculate hashes, parse artifacts, recover deleted files, inspect network traffic, and organize case information. Tools do not replace judgment. Their output must be validated, understood, and interpreted in the context of the case.

Use a controlled laboratory environment for practice. Work with copies of training images rather than original media, record the tool version and settings, and retain enough notes for another reader to reproduce the result. The chapters in this book introduce Linux utilities, Autopsy, The Sleuth Kit, Wireshark, regular expressions, data carving, and image-steganography techniques through that practical lens.

1.12 How this book is organized

The reader guide explains the recurring learning features and lab conventions; this roadmap explains the intellectual progression of the book. The chapters are designed as connected stages of an investigation rather than independent tool tutorials. The following roadmap shows the question each chapter helps an investigator answer.

Chapter	Connection to the forensic workflow
Chapter 3	Identify how computer-system layers, storage layout, and device context shape the available evidence.
Chapter 2	Interpret raw values, timestamps, encodings, hashes, and byte order accurately.
Chapter 4	Navigate a forensic workstation and inspect evidence with foundational Linux tools.

Chapter 5	Use more advanced Linux commands to make examination and analysis more efficient.
Chapter 6	Examine a USB image through Autopsy and connect artifacts to investigative questions.
Chapter 7	Perform transparent command-line file-system analysis with The Sleuth Kit.
Chapter 8	Preserve removable-media evidence by creating and verifying a forensic image.
Chapter 9	Search large evidence collections precisely with regular expressions.
Chapter 10	Recover content when file-system metadata is missing, deleted, or damaged.
Chapter 11	Analyze network evidence by capturing, filtering, and examining communications.
Chapter 12	Extract image evidence from captured network traffic.
Chapter 13	Identify and investigate data concealed inside digital images.
Chapter 14	Consolidate the techniques and plan the next steps in forensic practice.

The roadmap also explains the order of the book. Chapter 2 provides the representation skills used throughout the later analyses. The Linux chapters build the working environment; the acquisition and forensic-tool chapters establish preservation and examination workflows; and the final chapters apply those foundations to files, network traffic, and concealed data.

As you work through the book, keep three questions in mind: What does this artifact show? How reliable is that interpretation? What additional evidence would confirm or challenge it? Those questions turn tool output into defensible forensic reasoning.

1.13 Summary

Digital forensics applies repeatable methods to identify, preserve, examine, analyze, and report digital evidence. It complements cybersecurity and incident response by establishing reliable facts, documenting the work performed, and

separating direct observations from supported inferences. Effective investigations protect evidence integrity, respect legal and ethical scope, and correlate multiple artifacts before reaching conclusions.

Chapter Review

Review Questions

1. What is the difference between a direct forensic observation and an inference?
2. How do cybersecurity, incident response, and digital forensics differ in their primary purposes?
3. Why are hashes and chain-of-custody records important when handling evidence?
4. Name three common sources of digital evidence and one limitation of each.
5. Why can a running system require different collection decisions than powered-off media?

Discussion Questions

1. When should a responder prioritize volatile-data collection over immediate shutdown? What risks must be documented?
2. How can an investigator minimize privacy intrusion while still answering an authorized investigative question?

Hands-On Exercise

Exercise. Using the USB-and-network scenario in this chapter, create a one-page evidence plan. State the investigative question, list the likely evidence sources, identify which source should be preserved first, describe how integrity will be verified, and distinguish two observations from two possible inferences. Do not collect data from systems without authorization.

2 Number Systems for Digital Forensics

Learning Objectives

- Convert values among binary, decimal, hexadecimal, and octal representations.
- Interpret ASCII and Unicode values found in forensic artifacts.
- Convert UNIX timestamps to human-readable time and identify timezone considerations.
- Explain how hash values support integrity verification.
- Recognize how byte order affects multi-byte values.

Digital forensics depends on translating machine representations into defensible human interpretation. Investigators routinely encounter values stored as binary, hexadecimal, encoded text, UNIX timestamps, hash digests, and multi-byte fields inside low-level artifacts. To interpret those values correctly, they need to understand both what the representations mean and how to convert them quickly and accurately.

This chapter introduces the foundational representations that appear when reading command output, hex dumps, timestamps, file structures, and memory artifacts. We begin with number systems such as binary (base-2), decimal (base-10), octal (base-8), and hexadecimal (base-16), then connect those ideas to ASCII and Unicode text encoding, UNIX time, hash values, and byte order.

Mastering these representations is not just theoretical. It helps analysts explain what raw bytes mean, verify whether a value has been interpreted correctly, and move between machine-oriented and human-readable views of the same evidence. The chapter is organized around four practical threads: converting between number systems, translating between characters and their encoded values,

interpreting time values stored as numbers, and understanding how systems store and verify data at a low level. The following sections use Linux commands and Python examples to show how these ideas appear in day-to-day investigations.

This chapter applies the evidence-interpretation principles introduced in Chapter 1. Later chapters rely on these foundations when they examine file-system metadata, calculate and verify hashes, reconstruct timelines, and interpret network and storage artifacts.

2.1 Number system conversions in digital forensics

In digital forensics, converting between different number systems is a fundamental skill. Investigators frequently encounter data represented in binary or hexadecimal formats, while human interpretation often relies on the familiar decimal system.

This section will delve into the practical techniques for performing these conversions using both Linux command-line tools and Python scripting, providing forensic analysts with the means to efficiently translate and analyze digital evidence.

2.1.1 Linux-Based Number System Conversions

The Linux shell is often the fastest place to test an interpretation while examining logs, hex dumps, or memory values. The commands in this subsection help an investigator move quickly between the number systems most commonly encountered in forensic work.

Binary to decimal. Figure 2.1 summarizes the positional-weight method behind binary conversion, where each bit contributes a power of 2.

For example, the binary number ‘1011’ equals decimal 11 because $1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = 8 + 0 + 2 + 1$. Figure 2.2 shows the shell shortcut `echo $((2#101010))`, which is useful when the analyst wants an immediate decimal interpretation of a binary value.

The shell evaluates the arithmetic expression first and then `echo` prints the result.

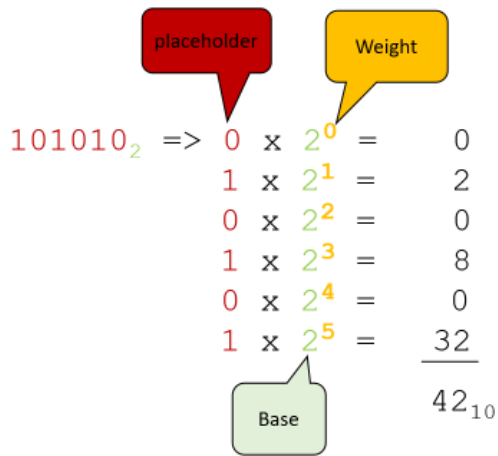


Figure 2.1: Illustration of binary to decimal conversion



Figure 2.2: Converting a binary number to its decimal equivalent

The output ‘42’ confirms that the binary value ‘101010’ corresponds to decimal 42. The important syntax is:

`$((...))` The shell arithmetic-expansion syntax.
`2#` The base prefix telling the shell to read the following digits as binary.
`echo` Prints the converted value so it can be compared with evidence.

Hexadecimal to decimal. Hexadecimal values appear constantly in forensic tools because they compactly represent binary bytes. Figure 2.3 shows the place-value method behind the conversion.

$$\begin{array}{rclclcl} \text{AB2}_{16} & \Rightarrow & (?)_{10} & & & \\ 2 \times 16^0 & = & 2 \times 1 & = & 2 & \\ \text{B} \times 16^1 & = & 11 \times 16 & = & 176 & \\ \text{A} \times 16^2 & = & 10 \times 256 & = & 2560 & \\ & & & & \hline & & & & 2738_{10} \end{array}$$

Figure 2.3: Illustration of hexadecimal to decimal conversion

For the hexadecimal value ‘AB2’, the decimal calculation is $10 \times 16^2 + 11 \times 16^1 + 2 \times 16^0 = 2560 + 176 + 2 = 2738$. Figure 2.4 demonstrates two shell commands that perform the same conversion.

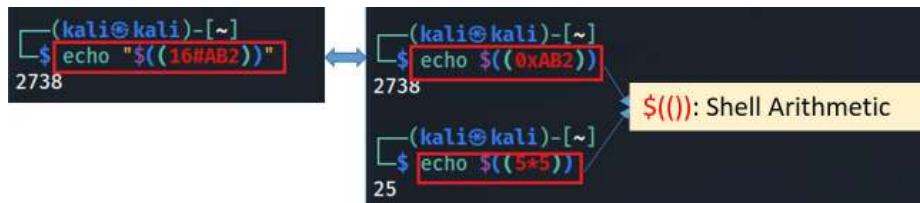


Figure 2.4: Demonstrating hexadecimal to decimal conversion

Using shell arithmetic. The command `echo ${16#AB2}` tells the shell to interpret ‘AB2’ as a base-16 number and print its decimal value. An equivalent form, `echo ${0xAB2}`, uses the more familiar hexadecimal prefix seen in many tools and programming languages. In each case, the key output to notice is ‘2738’. The forms differ only in notation: `16#AB2` is an explicit base prefix, while `0xAB2` is the more familiar hexadecimal prefix seen in many tools and

programming languages.

Decimal to binary. Investigators sometimes need to move the other way, especially when checking bit flags or permissions. Figure 2.5 shows the repeated-division-by-2 method that produces binary digits from a decimal number.

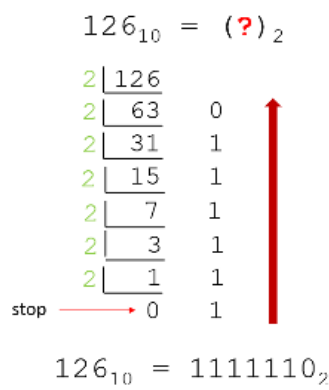


Figure 2.5: Illustration of decimal to binary conversion

The shell example in Figure 2.6 uses `echo "obase=2; 126" | bc` to produce the binary value '1111110'.

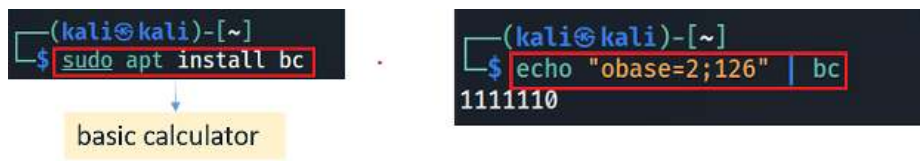


Figure 2.6: Linux command that converts decimal to binary

Using bc. This command is helpful when the investigator has a decimal value from a report or log and needs its binary form without doing the calculation manually. The pipe sends the text 'obase=2; 126' into `bc`, an arbitrary-precision calculator. Within that input, `obase=2` sets the output base to binary, and `126` is the decimal number being converted. The important output is '1111110', the binary equivalent of 126.

Decimal to hexadecimal. Figure 2.7 shows the manual repeated-division method for converting decimal numbers into hexadecimal values.

$$2738_{10} = (?)_{16}$$

16	2738	
16	171	2
16	10	11 = B
	0	10 = A

$$2738_{10} = AB2_{16}$$

Figure 2.7: Illustration of decimal to hexadecimal conversion

For the decimal value 2738, repeated division by 16 yields the hexadecimal digits A, B, and 2, producing ‘AB2’. Figure 2.8 shows two Linux methods for doing that same conversion directly.

```
(kali㉿kali)-[~]
$ echo "obase=16;2738" | bc
AB2
```

```
(kali㉿kali)-[~]
$ printf "%x\n" 2738
ab2
```

controls the output as in C `printf`

- `%d` - Print the argument as a signed decimal integer.
- `%x` - Print the argument as an unsigned hexadecimal integer.
- `%s` - Print the argument as a string.

Figure 2.8: Showcasing decimal to hexadecimal conversion

Using `bc`. The command `echo “obase=16; 2738” | bc` sends an instruction to `bc` to switch its output base to hexadecimal and then evaluate the decimal number 2738. The result to notice is ‘AB2’.

Using `printf`. The command `printf“%x\n”2738` formats the decimal number 2738 as hexadecimal and prints ‘ab2’. The `%x` specifier means “render this number in hexadecimal,” and `\n` adds a line break so the shell prompt stays readable. Investigators will often see lowercase hexadecimal in command output; the important point is the value, not the letter case.

Binary to hexadecimal. Investigators often move between binary and hexadecimal because hex dumps are more compact than long binary strings while still preserving exact byte values. Figure 2.9 shows the standard manual method: group the binary digits into sets of four, starting from the right, and convert each group to its hexadecimal equivalent.

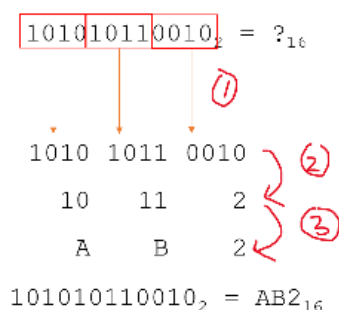


Figure 2.9: Illustration of binary to hexadecimal conversion

For example, the binary value ‘101010110010’ can be grouped as ‘1010 1011 0010’. These groups correspond to hexadecimal digits A, B, and 2, so the full hexadecimal value is ‘AB2’. This shortcut is especially useful when relating raw binary values in memory to the hexadecimal representations commonly displayed by forensic tools.

2.1.2 Python-Based Number System Conversions

Python is useful when these conversions need to become part of a repeatable script rather than a one-off shell check. Before using the examples, Figure 2.10 confirms that Python 3 is available on the system.

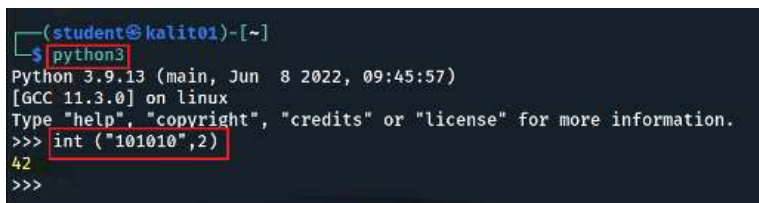
```
(student@kalit01)-[~]
$ python --version
Python 2.7.18

(student@kalit01)-[~]
$ python3 --version
Python 3.9.13
```

Figure 2.10: Verifying the installed Python versions

The version-check commands in Figure 2.10 report which Python interpreters are installed and which version number each one provides. The key thing to confirm is that Python 3 is available, because the examples in this chapter rely on Python 3 behavior and syntax.

Binary to decimal with `int()`. Figure 2.11 demonstrates `int("101010", 2)`, which tells Python to interpret the string '101010' as a base-2 value and convert it to an integer.

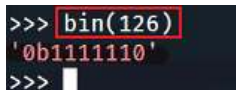
A terminal window with a dark background. The prompt is '(student@kalit01)-[~]'. The user enters 'python3'. The prompt changes to 'Python 3.9.13 (main, Jun 8 2022, 09:45:57) [GCC 11.3.0] on linux'. The user enters 'Type "help", "copyright", "credits" or "license" for more information.'. The user enters '>>> int("101010",2)'. The output is '42'. The user enters '>>>' again.

```
(student@kalit01)-[~]  
$ python3  
Python 3.9.13 (main, Jun 8 2022, 09:45:57)  
[GCC 11.3.0] on linux  
Type "help", "copyright", "credits" or "license" for more information.  
>>> int("101010",2)  
42  
>>>
```

Figure 2.11: Python command that converts binary (101010) to decimal (42)

The important parts are the binary string itself and the second argument, **2**, which specifies the base. The output '42' confirms the decimal interpretation.

Decimal to binary with `bin()`. Figure 2.12 shows the inverse operation with `bin(126)`.

A terminal window with a dark background. The user enters '>>> bin(126)'. The output is ''0b1111110''. The user enters '>>>' again.

```
>>> bin(126)  
'0b1111110'  
>>>
```

Figure 2.12: Convert decimal number to its binary equivalent in Python

Python returns '0b1111110'. The **0b** prefix marks the value as binary, and the remaining digits are the actual bit pattern. In other words, `bin()` returns a text-style representation of the number in base 2, which makes it convenient for quick checks, logging, or display inside larger forensic scripts.

2.2 ASCII: Bridging human-readable text and binary code

Beyond numerical conversions, investigators often need to determine whether raw bytes represent readable text, control characters, or encoded data. That makes character encoding a practical forensic skill rather than just a computing concept. **ASCII** (American Standard Code for Information Interchange) serves as

the bridge between human-readable characters and their underlying binary representations. It assigns unique numerical values to letters, numerals, punctuation marks, and control characters, enabling computers to store and process text.

The command `man ascii`, as illustrated in Figure 2.13, provides a comprehensive reference to the ASCII character set. An investigator might use this command when interpreting text recovered from a hex dump or when checking the numeric value associated with a control character.

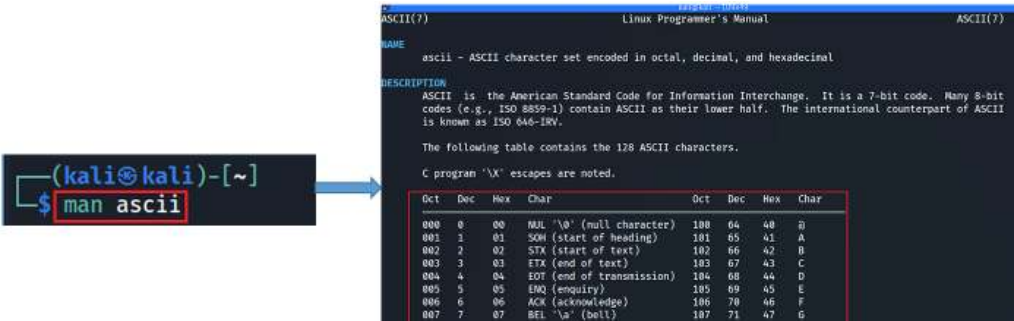


Figure 2.13: ASCII manual page in the terminal

The manual page output presents:

ASCII Table A tabular mapping between characters and their decimal, octal, and hexadecimal values.

Character Categories Explanations of control characters, printable characters, and other ASCII groupings.

In conclusion, proficiency in ASCII interpretation enables us to effectively analyze textual data, decipher hidden messages, and uncover crucial evidence embedded within the binary landscape of digital devices.

2.2.1 Understanding Character Encoding: ASCII vs. UTF-8

In the realm of digital forensics, understanding character encoding is vital, as it dictates how text data is represented and interpreted. **Unicode**, formally The “**Unicode Standard**” is an information technology standard for the consistent encoding, representation, and handling of text expressed in most of the world’s

writing systems.

Two common encoding standards are **ASCII** and **UTF-8**. While they both serve the purpose of representing characters using binary code, they differ significantly in their capabilities and applications.

ASCII. ASCII (American Standard Code for Information Interchange) remains important in digital forensics because many legacy artifacts, file headers, and control characters still rely on this limited character set.

- **Limited Character Set:** ASCII uses 7 bits to represent characters, allowing for a total of 128 unique codes. This is sufficient for basic English characters, numbers, and punctuation marks.
- **Simplicity:** Its simplicity and widespread adoption make ASCII a common choice for representing text in many legacy systems and file formats.
- **Inadequate for Non-English Text:** ASCII's limited character set makes it unsuitable for representing characters from other languages, leading to issues when dealing with multilingual data.

UTF-8. UTF-8 (Unicode Transformation Format - 8-bit) is more common in modern systems, so investigators need to recognize when text evidence extends beyond the ASCII range.

- **Extensive Character Set:** UTF-8 supports the entire Unicode character set, encompassing over 1 million characters from various languages and scripts worldwide.
- **Variable-Length Encoding:** UTF-8 employs a clever variable-length encoding scheme, using 1 to 4 bytes to represent different characters. This allows for efficient storage of both ASCII and non-ASCII characters.
- **Backward Compatibility:** UTF-8 is designed to be backward compatible with ASCII. The first 128 characters in UTF-8 have the same representation as in ASCII, ensuring seamless handling of basic English text.
- **Universal Adoption:** UTF-8's ability to handle a vast array of characters has led to its widespread adoption as the de facto standard for text encoding on the web and in modern computing systems.

With that context, we can move from character tables to practical conversions. In forensic work, these conversions help transform raw numeric values into readable text and, just as importantly, show when apparently readable text is actually

stored as hexadecimal bytes.

2.2.2 Linux ASCII Conversions

When an investigator sees suspicious bytes or recovered text fragments, Linux provides quick ways to translate between numeric values and readable characters. These conversions are especially helpful when validating what a hex dump or carved string actually represents.

Hexadecimal to ASCII with printf. Figure 2.14 uses the `printf` command with the hexadecimal byte sequence `\x48\x65\x6c\x6c\x66` to decode byte values into text.



Figure 2.14: Linux command that converts hexadecimal to ASCII

Each `\x` escape introduces one byte written as two hexadecimal digits, so `\x48` becomes 'H', `\x65` becomes 'e', and so on. The important output is 'Hello', which confirms that the recovered bytes decode to meaningful text.

Decimal to ASCII with nested printf. Figure 2.15 demonstrates `printf "\x$(printf "%x" "72")"`, which converts the decimal value 72 into the character 'H'.

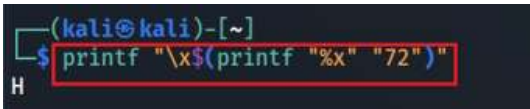
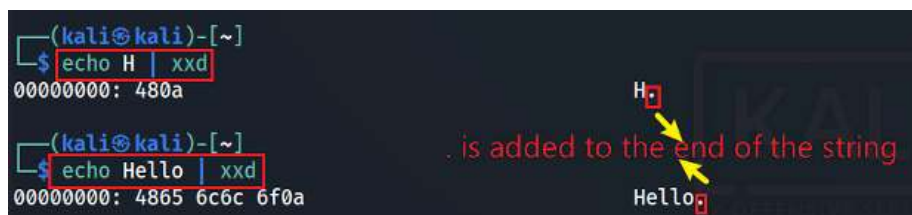


Figure 2.15: Decimal-to-ASCII conversion using nested printf command

The inner `printf "%x" "72"` converts decimal 72 into hexadecimal '48'. The `$(...)` syntax is command substitution: the shell runs the inner command first and inserts its output into the outer command. That means the shell effectively

builds `\x48`, and the outer `printf` then interprets that byte value and prints the corresponding character. This pattern is useful when evidence stores character values in decimal form but the shell expects byte-style hexadecimal escapes.

ASCII to hexadecimal with `xxd`. Figure 2.16 shows the reverse operation using `echo 'H' | xxd`.



```
(kali㉿kali)-[~]  
$ echo H | xxd  
00000000: 480a  
  
(kali㉿kali)-[~]  
$ echo Hello | xxd  
00000000: 4865 6c6c 6f0a
```

is added to the end of the string

Figure 2.16: Linux command that converts ASCII into hexadecimal

Here, `xxd` creates a hexadecimal dump of the incoming bytes. The output `'00000000: 480a'` shows that `'H'` is stored as `48`, while `0a` is the newline added by `echo`. If the investigator wants only the character bytes, `echo -n` avoids the trailing newline. The same approach works for longer text such as `'Hello'`, producing the byte sequence `48 65 6c 6c 6f`.

2.3 ASCII Conversions in Python 3

Continuing our exploration of character encoding transformations within Python, we now shift to Python 3 and its support for working directly with bytes. This matters when an investigator needs to decode recovered values, compare byte sequences, or feed converted text directly into the next parsing step of a script. In forensic scripts, this is often cleaner than shell pipelines because the decoded or encoded values can be reused immediately in later parsing steps.

The `binascii` module provides a straightforward bridge between raw bytes and hexadecimal text.

ASCII to hexadecimal with `hexlify()`. Figure 2.17 illustrates this conversion. The example call `binascii.hexlify(b'H')` converts a bytes object into its hexadecimal representation.

The `b` prefix in `b'H'` matters because it creates a bytes object rather than a

```
>>> import binascii
>>> binascii.hexlify(b'H')
b'48' ←
>>> binascii.hexlify(b'Hello')
b'48656c6c6f' ←
```

Figure 2.17: Demonstrating ASCII to hexadecimal conversion in Python 3

Unicode string. `hexlify()` then returns the hexadecimal byte sequence, here shown as `b'48'`. The same pattern converts longer text such as `b'Hello'` into `b'48656c6c6f'`.

Hexadecimal to ASCII with `unhexlify()`. Figure 2.18 shows the reverse operation using `binascii.unhexlify('48')`.

```
>>> binascii.unhexlify('48')
b'H' ←
>>> binascii.unhexlify('48656c6c6f')
b'Hello' ←
```

Figure 2.18: Demonstrating hexadecimal to ASCII conversion in Python 3

`unhexlify()` takes the hexadecimal string, decodes each pair of digits into a byte, and returns the result as a bytes object. In this example, the output is `b'H'`. When the input is `'48656c6c6f'`, the output becomes `b'Hello'`. These byte-oriented results are especially useful when the next step in the script is parsing or comparing raw artifact data.

2.4 Understanding UNIX time: A numerical representation of time

Transitioning from character encoding, we now introduce another crucial concept in digital forensics: **UNIX time**. This system provides a standardized way to represent points in time, which is essential for interpreting and correlating timestamps across different systems and applications.

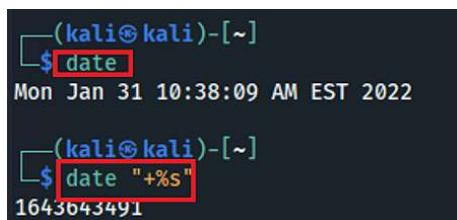
UNIX time, also known as “Epoch time” or “POSIX time”, represents the num-

ber of seconds that have elapsed since **January 1st, 1970 at 00:00:00** Coordinated Universal Time (UTC). This seemingly simple numerical representation serves as the backbone for timestamping in most modern operating systems and file systems.

In this section, we will delve into the practical aspects of working with UNIX timestamps in the Linux environment. We'll explore commands and techniques to convert between UNIX timestamps and human-readable date/time formats, empowering you to extract meaningful temporal information from digital evidence.

The Linux **date** command is often the quickest way to move between a readable time and its stored UNIX timestamp. This is useful when building a timeline from shell history, log entries, or file metadata.

Getting the current UNIX timestamp. Figure 2.19 shows **date “+%s”**, which prints the current time as the number of seconds since January 1, 1970 at 00:00:00 UTC.

A terminal window with a dark background. The prompt is (kali@kali)-[~]. The first command is \$ date, which outputs Mon Jan 31 10:38:09 AM EST 2022. The second command is \$ date +%s, which outputs 1643643491. Red boxes highlight the commands in both instances.

```
(kali@kali)-[~]  
$ date  
Mon Jan 31 10:38:09 AM EST 2022  
  
(kali@kali)-[~]  
$ date +%s  
1643643491
```

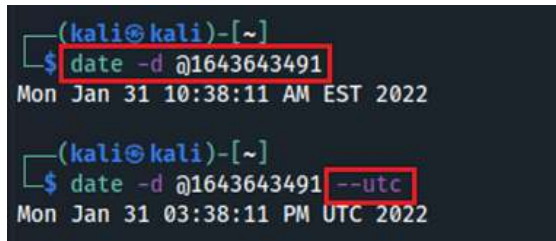
Figure 2.19: Date command to obtain the current UNIX timestamp

The key syntax is **+%s**. It tells **date** to format its output as epoch seconds rather than as a human-readable string. Before relying on this output in a forensic workflow, confirm that the system clock is accurate. If higher precision is needed, **%s.%N** adds nanoseconds.

Converting a UNIX timestamp to readable time. Figure 2.20 shows two versions of the conversion command.

Using local time. The command **date -d @1643643491** tells **date** to interpret the value after **@** as a UNIX timestamp in seconds. The output is shown in the system's local timezone.

Using UTC. The command **date -u -d @1643643491** converts the same times-



```
(kali㉿kali)-[~]  
$ date -d @1643643491  
Mon Jan 31 10:38:11 AM EST 2022  
  
(kali㉿kali)-[~]  
$ date -d @1643643491 --utc  
Mon Jan 31 03:38:11 PM UTC 2022
```

Figure 2.20: Converting UNIX timestamps to human-readable date and time format

tamp but forces the display into Coordinated Universal Time. The important lesson is that the numeric timestamp does not change; only the timezone used to display it changes. That distinction matters whenever the investigator is correlating artifacts from multiple systems.

2.5 UNIX Time (Python version)

Building upon our exploration of UNIX time in Linux, let's transition to Python, a versatile programming language frequently leveraged in digital forensics. This shift is useful when timestamp interpretation needs to become part of a repeatable workflow rather than a one-time terminal check. Python provides built-in modules and functions for handling UNIX timestamps, allowing for seamless integration of timestamp conversions and manipulations into investigative workflows.

In this section, we will delve into Python's capabilities for working with UNIX time, including converting between timestamps and human-readable formats, extracting specific date and time components, and performing calculations involving timestamps. By harnessing the power and flexibility of Python, forensic analysts can efficiently process and analyze the temporal aspects of digital evidence, gaining valuable insights into the timeline of events.

2.5.1 Getting Current Time and Timestamps

Python is a good fit when timestamp work needs to scale beyond a single shell command. The examples in this subsection show how to capture the current time

in both readable and numeric forms so it can be logged, compared, or reused in a script.

Using `datetime.now().timestamp()`. Figure 2.21 demonstrates `datetime.now().timestamp()`, which returns the current local time as a UNIX timestamp.

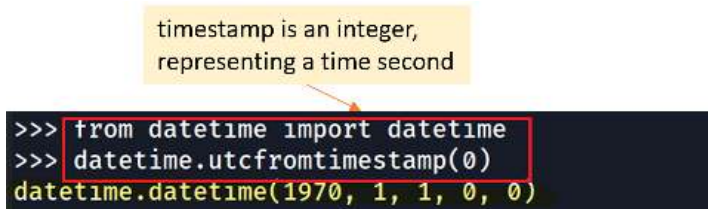


Figure 2.21: Obtaining the current UNIX timestamp

The import `from datetime import datetime` provides access to the `datetime` class. The call `datetime.now()` gets the current local date and time. The chained method `.timestamp()` then converts that value into seconds since the UNIX epoch. The result is usually a floating-point value, so fractional seconds are preserved.

Comparing local time and UTC. Figure 2.22 extends that idea by showing `datetime.now()` alongside `datetime.now(timezone.utc)`.

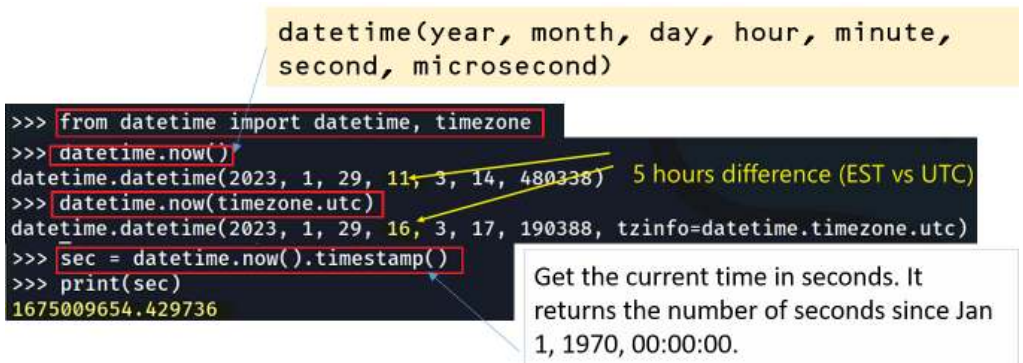


Figure 2.22: Demonstrating the use of the `datetime` module to obtain the current time in local and UTC timezones, as well as convert it to a UNIX timestamp

The important lesson is that Python can represent the same moment either as local time or as a timezone-aware UTC value. That distinction matters when evidence from multiple systems must be normalized before comparison.

Using the time module. Figure 2.23 shows the lighter-weight **time** module alternative.

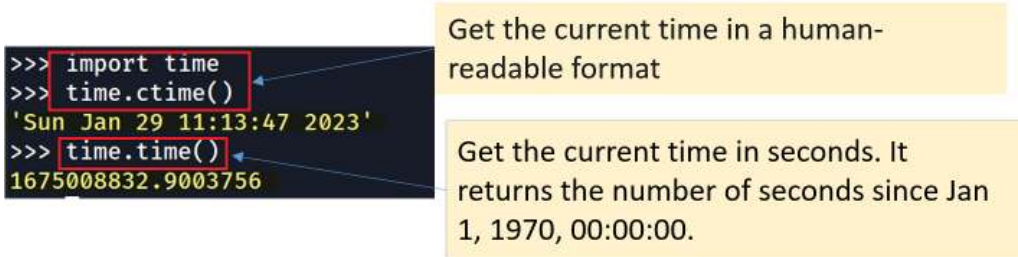


Figure 2.23: Python code using the time module to obtain the current time in both human-readable format and as a UNIX timestamp

`time.ctime()` returns the current local time as a readable string, while `time.time()` returns epoch seconds. This is convenient when a script needs only a quick timestamp and not a full **datetime** object.

2.5.2 Converting Timestamps to Readable Time

Forensic artifacts often store time as a raw number, so the next step is usually to make that value readable without losing track of timezone assumptions.

Using datetime.fromtimestamp(). Figure 2.24 shows this conversion step. The method `datetime.fromtimestamp()` converts a UNIX timestamp into a local **datetime** object.

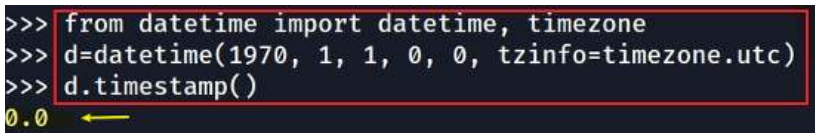


Figure 2.24: Converting a UNIX timestamp to a human-readable datetime object

This is the most direct way to turn a recovered UNIX value into something an investigator can place on a timeline. The key output is the readable date and time, but it should be interpreted as local time unless the code explicitly specifies otherwise.

Converting recovered timestamps. Figure 2.25 applies the same conversion pattern to a specific evidence-style value. In this example, the timestamp is 1675008832.

```
>>> datetime.utcfromtimestamp(1675008832)
datetime.datetime(2023, 1, 29, 16, 13, 52)
```

our default time zone is EST. Pay attention to the 5 hours difference

Figure 2.25: Converting a UNIX timestamp to a human-readable datetime object

The output `datetime.datetime(2023,1,29,11,13,52)` is the local-time interpretation of that timestamp. This is a reminder that the stored UNIX value is timezone-neutral, while the displayed result depends on the conversion context.

Using `datetime.utcfromtimestamp()`. Figure 2.26 shows the UTC-specific conversion with `datetime.utcfromtimestamp(1675008832)`.

If the timestamps are in UTC timezone (a common way to store dates) you should use

```
>>> datetime.utcfromtimestamp(1675008832)
datetime.datetime(2023, 1, 29, 16, 13, 52)
```

our default time zone is EST. Pay attention to the 5 hours difference

Figure 2.26: Demonstrating the conversion of a UNIX timestamp to a UTC date-time object

This form is useful when building a normalized cross-system timeline, because it returns the timestamp as a UTC datetime rather than as local time. For evidentiary interpretation, that explicitness is often safer than relying on the host system’s timezone settings.

2.5.3 Comparing Epochs and Time Differences

Some investigations require more than display conversion. They require comparing two timestamp systems and calculating the offset between them.

How the Script Works. Figure 2.27 explains a small Python script, `time_diff.py`, that calculates the offset between the UNIX epoch (January 1, 1970) and the iOS Cocoa Core Data epoch (January 1, 2001).

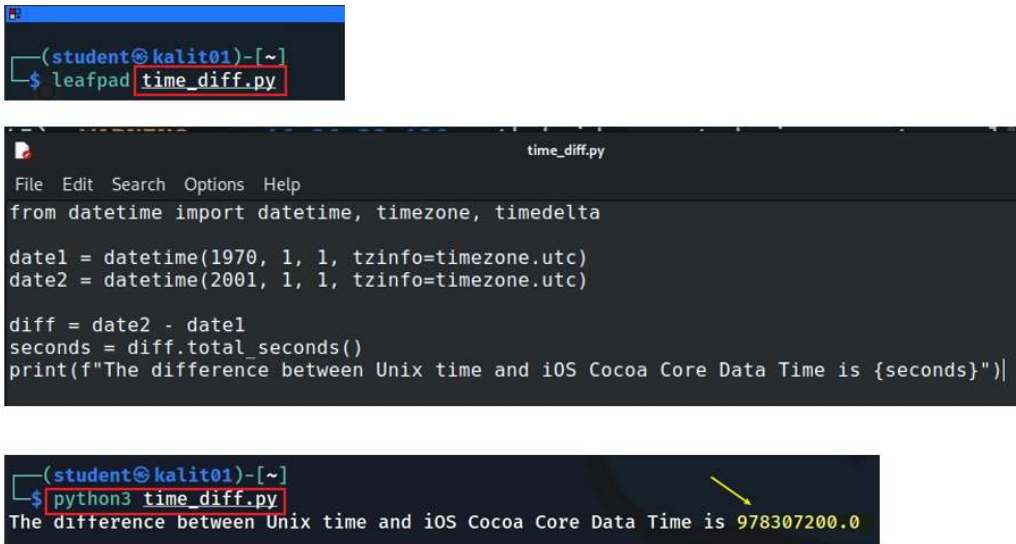


Figure 2.27: Calculating the time difference between UNIX epoch and iOS Cocoa Core Data time reference

The script imports `datetime`, `timezone`, and `timedelta`, creates the two reference dates in UTC, subtracts them, and then calls `total_seconds()` on the resulting `timedelta`. The important output is **978,307,200 seconds**, which is the offset an investigator must account for when translating between these two timestamp systems.

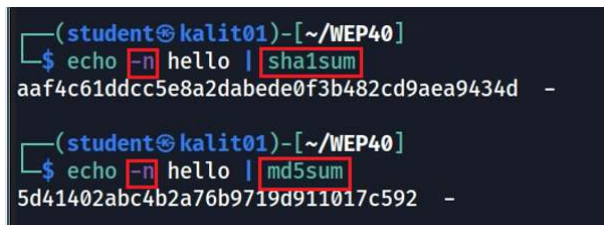
2.6 Understanding hash codes in investigations

Moving beyond character encoding and timestamp analysis, another critical aspect of digital forensics involves verifying the integrity and authenticity of data. Investigators rely on hashes when documenting evidence, checking whether files changed, and comparing recovered data against known values. **Hash functions** play a pivotal role in this regard, providing a mechanism to generate unique digital fingerprints for files or data, irrespective of their size or complexity.

A **hash function** takes an input (data or file) and produces a fixed-size output, known as a hash value or digest. This hash value acts as a unique identifier for the input data. Even the slightest change in the input will result in a drastically different hash value, making hash functions invaluable for ensuring data integrity and facilitating comparisons.

2.6.1 Command-Line Hashing

Hash functions, as discussed previously, provide a mechanism to generate unique digital fingerprints for files or data. The terminal commands demonstrated in Figure 2.28 showcase the use of two widely-used hashing algorithms, “SHA-1” and “MD5”, to compute hash codes for the string ‘hello’ in a Linux environment.

A terminal window with a dark background and light-colored text. The prompt is `(student@kalit01)-[~/WEP40]`. The first command is `$ echo -n hello | sha1sum`, and the output is `aaf4c61ddcc5e8a2dabede0f3b482cd9aea9434d -`. The second command is `$ echo -n hello | md5sum`, and the output is `5d41402abc4b2a76b9719d911017c592 -`.

```
(student@kalit01)-[~/WEP40]
$ echo -n hello | sha1sum
aaf4c61ddcc5e8a2dabede0f3b482cd9aea9434d -

(student@kalit01)-[~/WEP40]
$ echo -n hello | md5sum
5d41402abc4b2a76b9719d911017c592 -
```

Figure 2.28: Calculating SHA-1 and MD5 hashes for the string “hello”

Using SHA-1. The command `echo -n hello | sha1sum` prints the string ‘hello’ without a trailing newline and pipes those bytes into `sha1sum`. The important output is the digest `aaf4c61ddcc5e8a2dabede0f3b482cd9aea9434d`. The

trailing hyphen indicates that the bytes came from standard input rather than from a named file.

Using MD5. The command `echo -n hello | md5sum` follows the same pattern but uses `md5sum` instead. The important output is `5d41402abc4b2a76b9719d911017c592`. In both examples, the pipe passes the evidence bytes directly to the hash program, and the digest is the part an investigator would compare or document.

The ability to generate and compare hash codes is a fundamental skill for digital forensic analysts. The `sha1sum` and `md5sum` commands, as showcased in Figure 2.28, provide readily available tools for calculating hashes in a Linux environment. While these specific algorithms might have limitations in terms of cryptographic strength, they remain relevant in various forensic scenarios for verifying data integrity and facilitating evidence identification.

2.6.2 Python Hashing

In addition to command-line tools, Python provides robust capabilities for calculating hash codes, a critical aspect of digital forensics for verifying data integrity and identifying files. Figure 2.29 showcases the use of Python’s **hashlib** module to generate “SHA-1” and “MD5” hashes for the string “hello”.

Using MD5 in Python. The call

```
hashlib.md5("hello".encode('utf-8')).hexdigest()
```

follows the same pattern and returns `5d41...c592`. The key idea is that once the data is in bytes, the same workflow can be reused for stronger algorithms such as SHA-256.

While **MD5** and **SHA-1** are illustrated here, it’s important to remember that **SHA-256** or stronger algorithms are generally recommended for modern forensic applications due to the increasing vulnerabilities of older hash functions.


```
1 import hashlib
2 hashlib.sha1("hello".encode('utf-8')).hexdigest()
3
✓ 0.0s

'aaf4c61ddcc5e8a2dabede0f3b482cd9aea9434d'

1 hashlib.md5("hello".encode('utf-8')).hexdigest()
✓ 0.0s

'5d41402abc4b2a76b9719d911017c592'
```

Figure 2.29: Calculating SHA-1 and MD5 hashes for a string “hello”

2.7 The significance of byte order in digital forensics

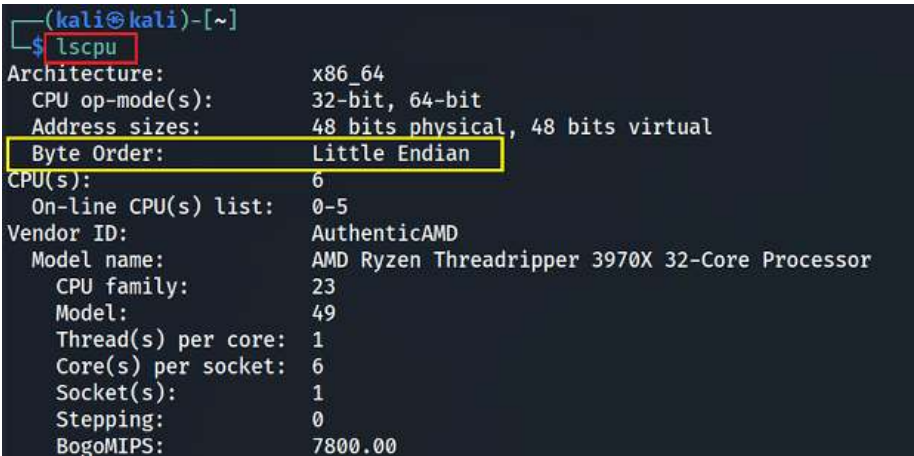
In digital forensics, a crucial aspect of data analysis lies in understanding how data is organized and stored in memory, particularly when dealing with multi-byte values. Misreading byte order can completely change the meaning of a timestamp, file signature, length field, or integer recovered from raw evidence. This arrangement, known as byte order or endianness, directly impacts how data is interpreted and can significantly influence the accuracy of forensic investigations.

While the previous sections have focused on the representation and conversion of numerical data, this section highlights a critical aspect of how data is stored in memory: byte order, also known as endianness.

2.7.1 Byte order identification

Figure 2.30 presents the output of the `lscpu` command. An investigator might run `lscpu` to confirm how the current system stores multi-byte values before interpreting raw bytes from memory, disk structures, or carved data. The command reports several CPU characteristics, but for this discussion the key line to

notice is **Byte Order**. If the output says **Little Endian**, the least significant byte is stored first in memory; if it says **Big Endian**, the most significant byte is stored first. That single line gives the reader the information needed to interpret multi-byte values correctly in later forensic analysis.



```
(kali@kali)-[~]  
$ lscpu  
Architecture:          x86_64  
CPU op-mode(s):        32-bit, 64-bit  
Address sizes:          48 bits physical, 48 bits virtual  
Byte Order:             Little Endian  
CPU(s):                 6  
On-line CPU(s) list:    0-5  
Vendor ID:              AuthenticAMD  
Model name:             AMD Ryzen Threadripper 3970X 32-Core Processor  
CPU family:             23  
Model:                  49  
Thread(s) per core:     1  
Core(s) per socket:     6  
Socket(s):              1  
Stepping:               0  
BogoMIPS:               7800.00
```

Figure 2.30: Output of the lscpu command showing the system’s byte order

2.7.2 Contrasting byte ordering schemes: Little Endian vs. Big Endian

As demonstrated in the previous output (Figure 2.30), the line “Byte Order: Little Endian” reveals that the system under investigation utilizes the little-endian format.

The concept of endianness as demonstrated in Figure 2.31 revolves around how the bytes within a multi-byte data type (such as integers or floating-point numbers) are arranged in memory.

- **Little Endian:** In a little-endian system, the least significant byte (LSB), which holds the smallest positional value within the multi-byte data, is stored at the lowest memory address. As we see in Figure 2.31, the byte 0x04 (the LSB) occupies the address 0x100, while the most significant byte (MSB), 0x01, is stored at the highest address, 0x103. This arrangement might seem counter-intuitive to humans who typically read and write numbers from left to right (most significant



Figure 2.31: Illustration of little endian and big endian byte ordering

digit first).

- **Big Endian:** In contrast, big-endian systems prioritize the most significant byte (MSB), placing it at the lowest memory address. Figure 2.31 illustrates this by showing 0x01 (the MSB) at address 0x100 and 0x04 (the LSB) at 0x103. This ordering aligns more closely with our conventional understanding of numerical representation.

2.8 Summary

This chapter introduced the core data representations that investigators encounter when moving between raw evidence and human-readable interpretation. We examined how to convert values among binary, decimal, and hexadecimal; how to translate between bytes and text using ASCII and UTF-8 concepts; how to interpret UNIX timestamps and compare time systems; how to use hashes to document and verify integrity; and how byte order affects the meaning of multi-byte values.

Together, these skills support everyday forensic tasks such as reading hex dumps, validating encoded text, normalizing timestamps across systems, comparing evidence with known values, and correctly interpreting low-level structures. The Linux and Python examples in this chapter are meant to provide quick reference points that analysts can reuse when explaining findings, checking assumptions, and building repeatable workflows.

Chapter Review

Review Questions

1. Why is hexadecimal commonly used when examining raw digital evidence?
2. Convert the hexadecimal value 2A to decimal and binary.
3. What information does a UNIX timestamp represent, and why should its timezone be recorded?
4. What can matching hash values support, and what do they not prove?
5. How can incorrect byte-order interpretation change a forensic conclusion?

Discussion Questions

1. What safeguards should an analyst use before relying on a converted timestamp in a report?
2. Why should hash comparisons be combined with documentation rather than treated as a complete chain of custody?

Hands-On Exercise

Exercise. Using the Linux or Python examples already introduced in this chapter, convert 10110110, 7F, and decimal 255 into the other relevant number systems. Convert one UNIX timestamp to UTC and local time, calculate a hash for a practice file, and record the commands and results in an analyst note.

3 Computer Systems and Digital Evidence

Learning Objectives

- Describe the hardware, firmware, operating-system, application, and user layers of a computer system.
- Identify potential evidence sources associated with processors, memory, storage, network interfaces, and peripherals.
- Distinguish sectors, partitions, Master Boot Records, and GUID Partition Tables at a conceptual level.
- Explain the legacy BIOS/MBR and modern UEFI/GPT boot paths.
- Identify the boot and partition-table observations that should be documented during an examination.

Digital evidence is created by a system of connected layers, not by one application alone. A user's action can leave traces in an application, the operating system, memory, storage, network records, and attached devices. Before an examiner uses a forensic tool, it is helpful to understand where those traces originate and what each layer can and cannot show. This chapter provides that orientation for the later hands-on work in the book.

3.1 Computer systems as evidence environments

A computer system combines physical components with software that coordinates them. The hardware includes processors, memory, storage, network interfaces,

and peripheral devices. Firmware initializes hardware; the operating system manages resources; applications perform user-facing work; and users or automated processes create activity. Each layer can create useful artifacts, and the relationship among the layers helps an investigator test an explanation rather than rely on a single trace [Kent et al., 2006].

Figure 3.1 shows the four computer-system layers used throughout this chapter: user, applications, operating system, and device or computer hardware. A single user action can create related artifacts at more than one layer, so an examiner should use the figure as a guide to possible evidence sources rather than as a guarantee that a particular trace exists.

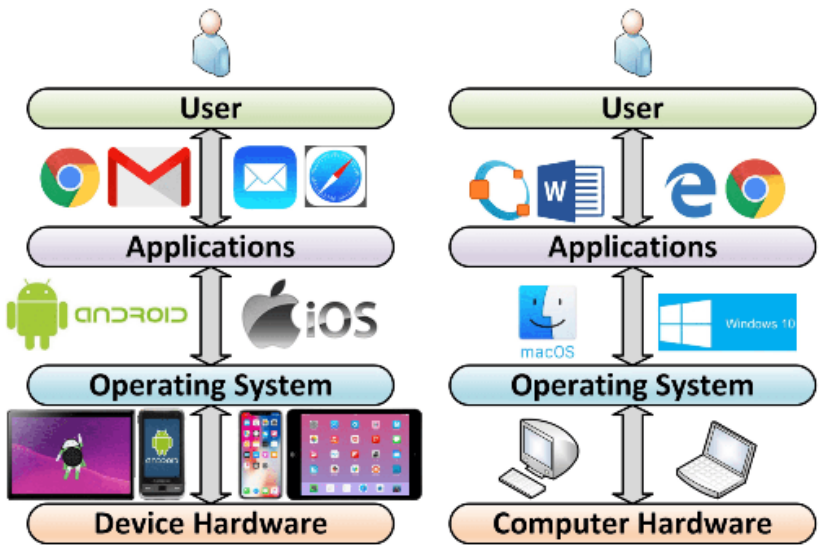


Figure 3.1: Computer-system layers. User activity can generate related evidence in applications, operating systems, and hardware.

Table 3.1: Computer-system layers and examples of the evidence each layer may generate.

Layer	Role	Examples of potential evidence
-------	------	--------------------------------

User	Initiates, directs, or automates activity through an account or interface.	Account activity, authentication records, input history, and actions attributed to a user context.
Applications	Create, view, transmit, and modify information for a user or service.	Documents, browser history, application logs, account activity, and recent-file lists.
Operating system	Schedules work and manages files, memory, devices, and network access.	Event logs, process information, user accounts, configuration data, and file metadata.
Device or computer hardware	Processes, retains, and communicates data through components such as memory, storage, network interfaces, and peripherals.	Memory contents, disk sectors, partitions, device serial numbers, network-interface information, and removable-media traces.

Table 3.1 uses the same four-layer model as the figure. It is a reasoning aid, not a guarantee that every artifact exists or is reliable. A browser record may indicate that a URL was stored in a profile, while network records may show that data was transmitted. Neither observation alone proves who was at the keyboard or why an action occurred. Stronger conclusions come from correlating artifacts across layers and documenting their limitations.

3.2 Core components and their forensic relevance

Before examining individual artifacts, students should be able to locate the major components of a computer system and explain their basic roles. Figure 3.2, adapted from the Carnegie Cyber Academy’s *How Computers Work* resource, identifies the CPU, memory, storage, network interface, motherboard, and con-

nected peripherals [Carnegie Cyber Academy, 2026]. In a forensic examination, these components are potential sources of distinct artifacts, not proof of a particular user action.

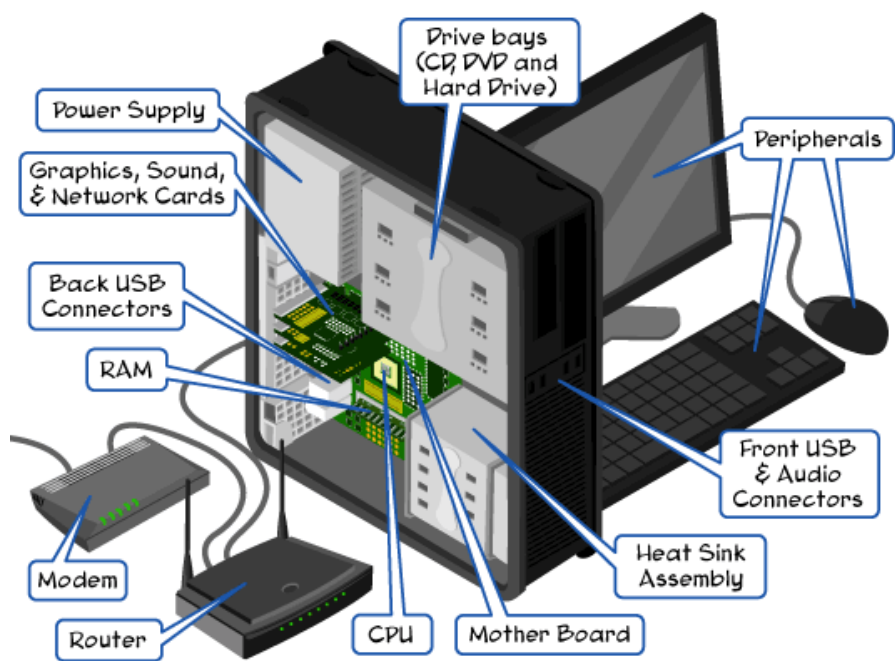


Figure 3.2: High-level computer components, including the CPU, RAM, motherboard, storage drive bays, network card, and peripherals. Reproduced with permission from Carnegie Cyber Academy, *How Computers Work* [Carnegie Cyber Academy, 2026].

Processor and memory. The central processing unit executes instructions, and memory holds data needed by running processes. Memory can contain volatile evidence such as active processes, network connections, decrypted material, and unsaved content. Because this information can disappear when power is removed, collection decisions for a running system require authorization, documented judgment, and attention to the risk of altering evidence.

Figure 3.3 shows a command-line system-information report. Its value is not the particular values on the screen, but the categories an examiner should record: operating-system and kernel information, processor architecture, memory, stor-

age, and network interfaces.

Figure 3.3: A command-line system-information report. Record the observed configuration and time of collection; do not infer user activity from configuration alone.

Storage. Hard drives, solid-state drives, USB devices, and memory cards retain data after power is removed. Their contents can include allocated files, deleted content, unallocated space, metadata, and system configuration. A forensic image captures storage at a point in time so that later examination can proceed from a working copy rather than the original media.

Network interfaces and peripherals. Network adapters, printers, cameras, phones, external drives, and other peripherals can expand the evidence environment. They may introduce identifiers, connection records, transferred content, or storage outside the primary computer. An examiner should record which devices are present and avoid assuming that an artifact on one device explains all activity across the system.

3.3 Runtime system context and evidence correlation

The components described above work together while a system is running. Figure 3.4 distinguishes a stored program from a process, its executing instance, and the threads through which that process performs work. The operating system schedules those threads on the processor, allocates memory, records relevant events, and coordinates access to storage, network interfaces, and peripheral devices.

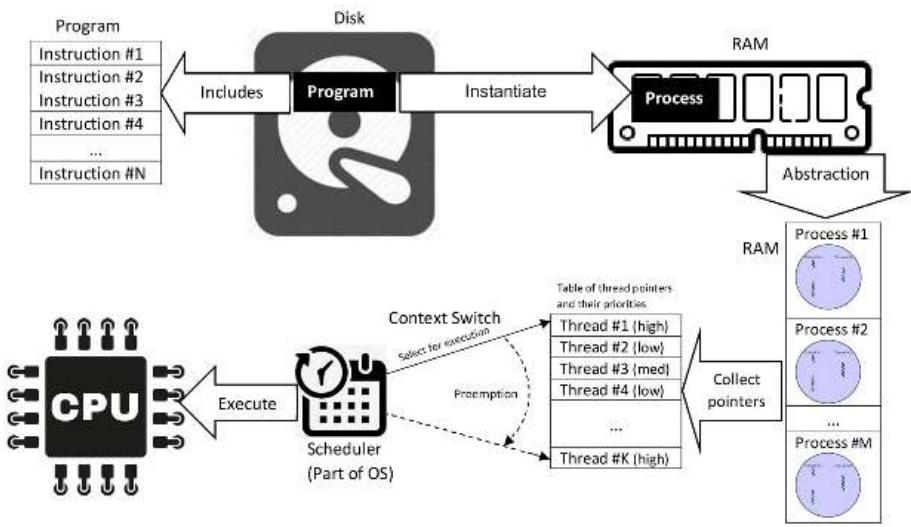


Figure 3.4: Program, process, and thread relationships. The relationship helps explain why execution, scheduling, and memory state can be valuable volatile evidence.

The process-monitoring view in Figure 3.5 provides a complementary runtime snapshot. Process names, process identifiers, parent-child relationships, and resource use can help guide an investigation, but the display is time-sensitive and requires corroboration from other artifacts.

The runtime model maps directly to several common forensic tasks:

Memory forensics. A RAM capture can preserve volatile processes, network

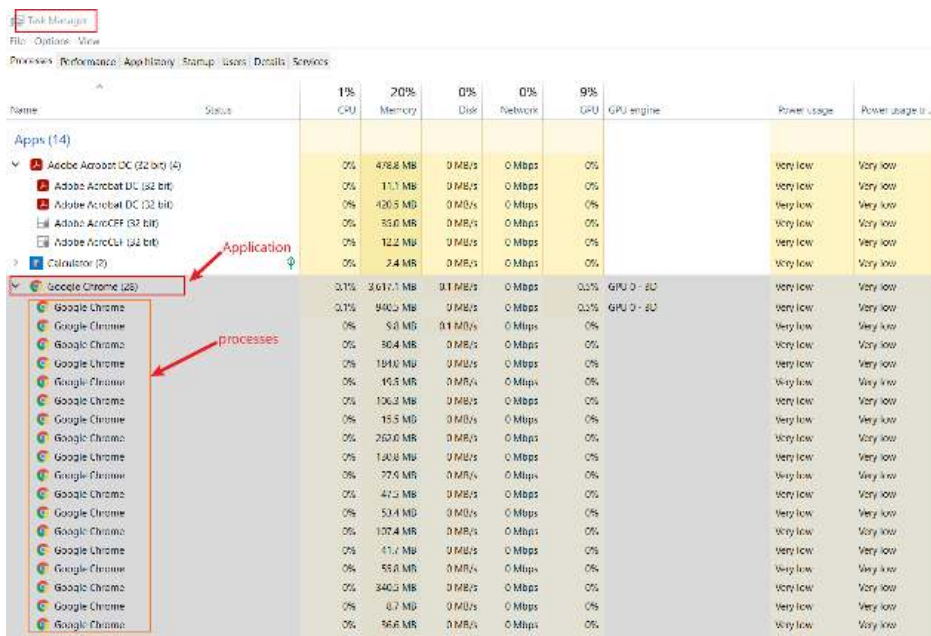


Figure 3.5: A process-analysis interface. Treat a process list as a snapshot of a running system and document the tool, time, and collection conditions.

connections, decrypted material, unsaved content, and sometimes injected code.

Process analysis. Process identifiers, parent-child relationships, command lines, loaded modules, and resource use can help reconstruct program execution.

Operating-system log analysis. Event records, authentication activity, service changes, and device connections provide system-level context.

Application-log and artifact analysis. Browser, email, office, and other application records may show user-facing activity and are often persisted to storage.

Static analysis. Inspect an executable without running it. Its metadata, hashes, strings, imports, and code structure can indicate potentially malicious capability.

Static program analysis is not a runtime task, but it complements process and memory analysis by connecting an observed executable to its possible behavior. Each task has limits and should be interpreted alongside other artifacts.

Consider a user who opens a browser and downloads a document. The application may create one or more processes and threads; the processor executes them and memory can hold transient content, connection details, or decrypted data. The operating system may record process, account, and event information; storage may retain the downloaded file, cache data, and metadata; and network records may show the connection or transfer. Each source answers a different part of the evidence question, so an examiner should correlate them rather than treat any one artifact as conclusive.

Artifact availability varies by system. Operating-system and application versions, cloud synchronization, encryption, hardware, and connected devices all affect what is retained. A missing artifact may reflect configuration or normal system behavior rather than deliberate concealment. Document the device type, operating-system version, relevant accounts, attached media, network state, collection time and timezone, and tools used. This record helps another examiner understand which conclusions are supported and which questions remain open.

3.4 Hard disks as forensic evidence sources

Hard disk drives (HDDs) store data on rotating magnetic platters. Figure 3.6 shows the internal components of a typical HDD: platters retain data, a spindle rotates them, and an actuator positions read/write heads over their surfaces. The electronics board and firmware coordinate those mechanical components with the host computer.



Figure 3.6: Internal components of a hard disk drive. Platters, the spindle, the actuator, and read/write heads provide physical context for sector-based evidence [Reg Media, 2006].

Figure 3.7 relates the visible drive components to the storage units used during examination. The platter is the magnetic surface that stores data; a track is a circular path on that surface; and a sector is an addressable portion of a track. The actuator moves the read/write heads to the required location. This geometry is a conceptual model: modern drives normally present logical block addresses (LBAs) rather than exposing a physical track-and-sector location. Nevertheless, sector numbers and byte offsets let an examiner describe where data was observed even when a filename is unavailable.

Physical and logical storage. A sector is the smallest addressable unit that

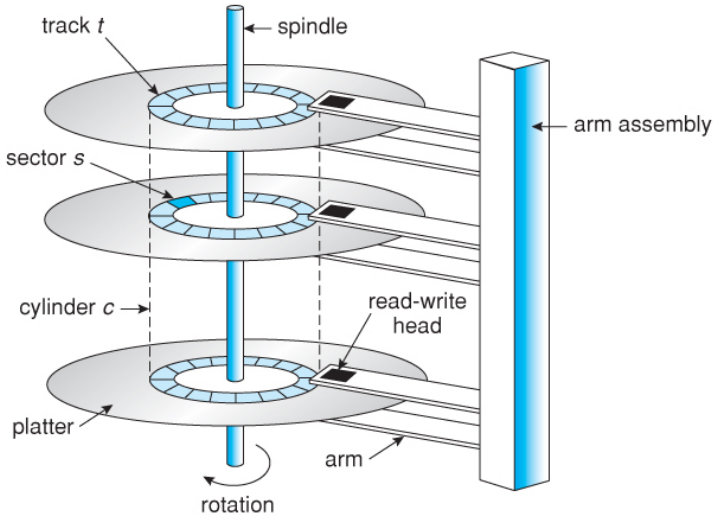


Figure 3.7: Hard-disk geometry. The labeled platter, track, sector, and read/write head provide context for sector- and offset-based examination [Silberschatz et al., 2013].

a drive normally exposes to the operating system. Many drives use 512-byte logical sectors, while others use 4,096-byte sectors or a 512-byte logical interface over 4,096-byte physical sectors. Operating systems present these sectors through logical structures, including partitions and volumes, which the next section introduces.

Capacity calculation. For a device with 81,600,000 sectors and 512 bytes per logical sector, multiply the sector count by the sector size:

$$81,600,000 \text{ sectors} \times 512 \text{ bytes/sector} = 41,779,200,000 \text{ bytes.}$$

Using binary multiples, 1 KiB = 1,024 bytes, 1 MiB = 1,024 KiB, and 1 GiB = 1,024 MiB. The capacity is therefore:

$$\begin{aligned} & 41,779,200,000 \text{ bytes} \div 1,024 \\ & = 40,800,000 \text{ KiB,} \quad 40,800,000 \text{ KiB} \div 1,024 = 39,843.75 \text{ MiB,} \\ & \quad 39,843.75 \text{ MiB} \div 1,024 \approx 38.91 \text{ GiB.} \end{aligned}$$

Drive manufacturers commonly use decimal units, where 1 GB = 1,000,000,000 bytes. The same value is therefore approximately 41.78 GB in decimal notation. Record both the sector count and the sector size in an evidence note; a displayed capacity alone does not identify the source device.

Figure 3.8 is a disk surface-scan interface. Its grid summarizes tested regions of the drive: green blocks indicate that the scan did not report a read error for that region, while red blocks would indicate a reported problem. The display does not show raw sector bytes, does not identify the contents of a region, and does not by itself verify a forensic image. It can, however, help an examiner document observed media condition alongside the device identity, scan settings, time, and any acquisition results.

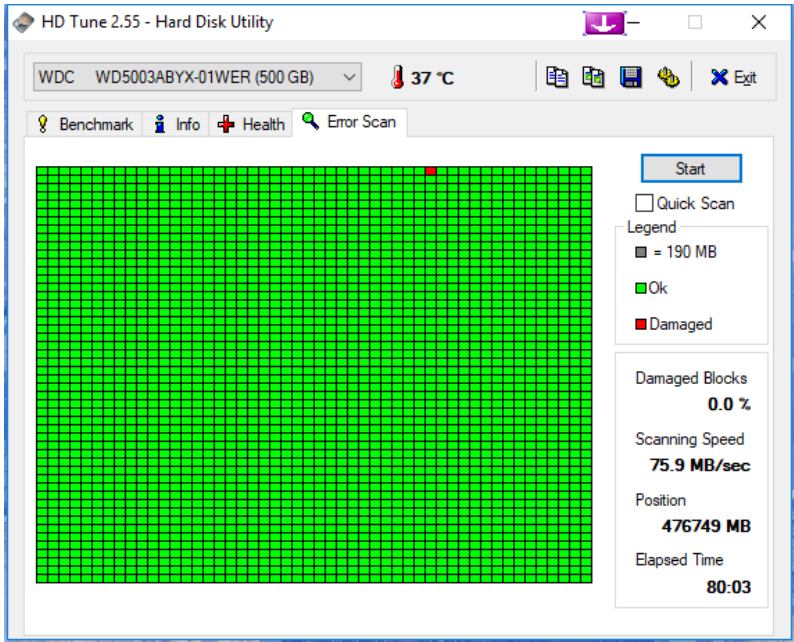


Figure 3.8: A disk surface-scan interface. The grid summarizes tested regions and reported read errors; it is not a raw-byte sector view or a verification of forensic-image integrity.

Forensic work normally examines a verified image rather than opening evidence media. At the logical level, an HDD can contain partitions, volumes, allocated files, deleted content, unallocated space, and metadata. At the physical level, the

structure explains why sector numbers and offsets can locate data independently of a filename. Damage, firmware problems, wear, and improper handling can affect what can be recovered, so preservation decisions should protect the original device and document its condition before examination.

Although solid-state drives use different physical technology, they present many of the same logical evidence concepts. Their controller behavior, wear leveling, and TRIM operations can make deleted-data recovery less predictable. Section 3.5 next examines the logical organization of storage.

3.5 Partitions, volumes, and drive letters

A partition divides a physical storage device into logical regions. A volume is storage made available to an operating system. A drive letter, such as **C:** or **D:** in Windows, is a convenient operating-system name for a mounted volume; it is not itself a physical disk. These distinctions help an examiner avoid confusing an entire source device with only one of its logical regions.

Figure 3.9 provides a conceptual view: one disk can contain multiple partitions, and each partition can provide a volume to the operating system. The illustration simplifies modern storage configurations, but it establishes the basic relationship needed to interpret later forensic views.

Figure 3.10 shows how Windows presents volumes to a user. The displayed names, drive letters, and free-space values are useful leads, but they do not establish the physical-device identity, partition boundaries, or forensic integrity of the data.

In Windows, right-click the Start button (or press **Win+X**) and choose **Disk Management**, as shown in Figure 3.11. Do not select **Device Manager**: it is useful for hardware-device information but does not provide the partition-and-volume view used in this section.

Figure 3.12 correlates physical disks with their partitions and volumes in Windows Disk Management. Read the disk number, capacity, partition size, status, and drive letter together. Forensic work should normally use an acquired image and a dedicated analysis tool; a live disk-management display is only a non-destructive laboratory orientation aid.

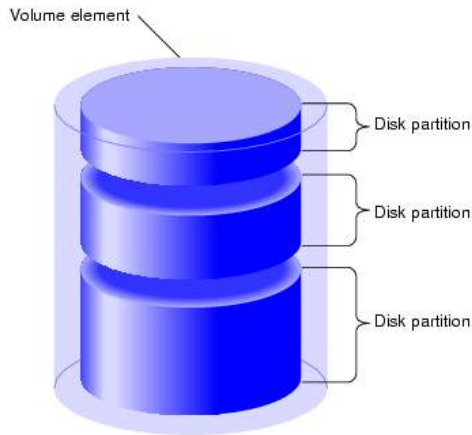


Figure 3.9: One physical disk divided into logical partitions. A partition can contain a volume that the operating system makes available to users.

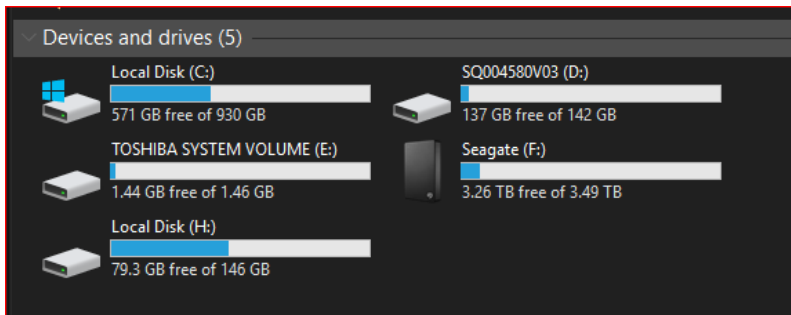


Figure 3.10: Windows Explorer displays mounted volumes with drive letters and available space. These labels are logical operating-system views, not physical-device identifiers.

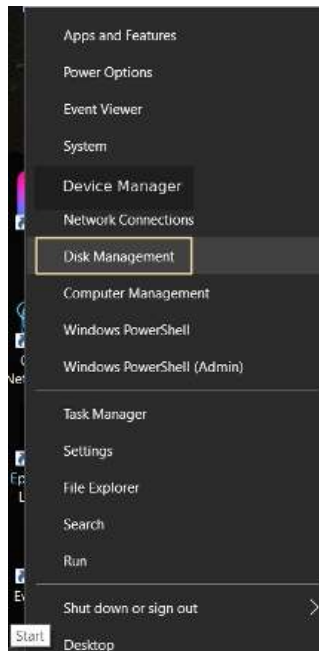


Figure 3.11: Opening Windows Disk Management from the Start-button context menu. Select **Disk Management**, not **Device Manager**, to view disks, partitions, and volumes.

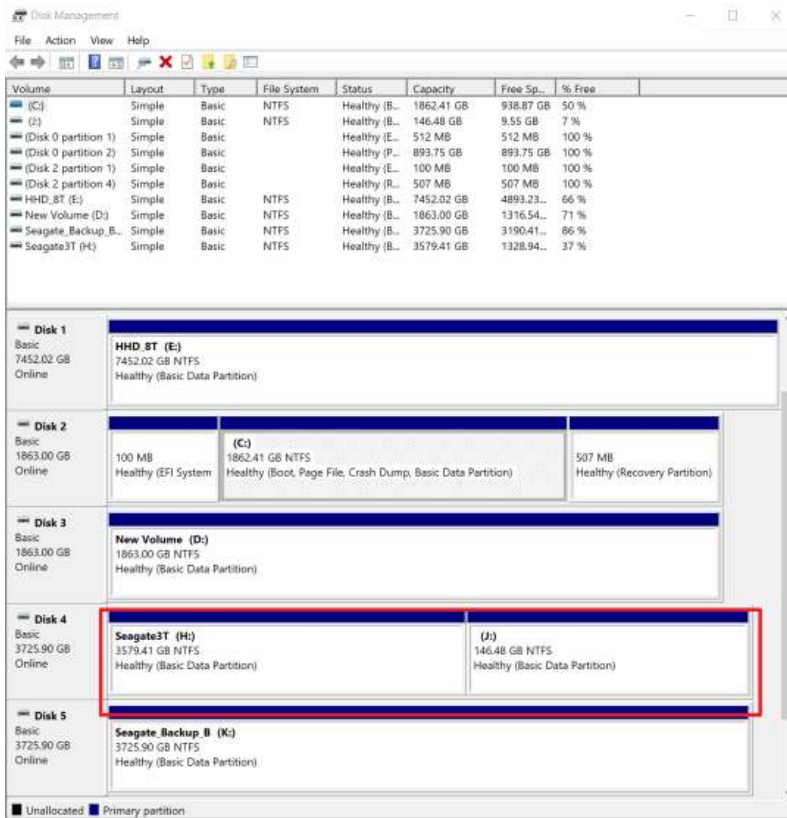


Figure 3.12: Windows Disk Management correlates physical disks, partitions, volumes, sizes, and drive letters. Use it only for authorized, non-destructive laboratory observation.

3.6 Booting process and partition-table overview

A partition table records how a device’s partitions are organized. The two common formats are the Master Boot Record (MBR) and GUID Partition Table (GPT). Table 3.2 provides a high-level comparison before the two structures are examined in detail. The firmware pairing is typical rather than absolute: systems can support compatibility modes, so an examiner should identify the actual firmware mode and partition structures from the evidence.

Table 3.2: High-level comparison of MBR and GPT partition tables [UEFI Forum, 2024, Microsoft, 2026a].

Partition table	Typical firmware pairing	Key characteristics
MBR	Legacy BIOS	Up to four primary partitions; practical 2 TiB limit with 512-byte logical sectors.
GPT	Modern UEFI	Primary and backup metadata; supports large disks; commonly 128 partition entries in Windows.

The boot process provides important context for an examination because it connects firmware settings, a selected storage device, boot code, and the running operating system. The exact sequence depends on whether a device uses the older BIOS/MBR approach or the modern UEFI approach, and on its operating-system configuration.

Legacy BIOS and MBR booting. After power is applied, firmware performs initial hardware checks, commonly called the power-on self-test (POST). In a traditional PC, the Basic Input/Output System (BIOS) initializes essential devices and follows its configured boot order. BIOS firmware was historically stored in ROM or flash memory, while its configuration was commonly stored in CMOS-backed memory. The firmware selects a boot device and transfers control to the boot code in that device’s MBR. The MBR code normally identifies an active partition and transfers control to its volume boot record, which begins the operating system’s boot-manager and loader sequence [Microsoft, 2026b].

Figure 3.13 illustrates this legacy hardware-level path from the power supply

through firmware and the CPU to a selected storage device. It is a conceptual BIOS/MBR model; modern UEFI systems do not normally bootstrap an operating system by executing MBR code.

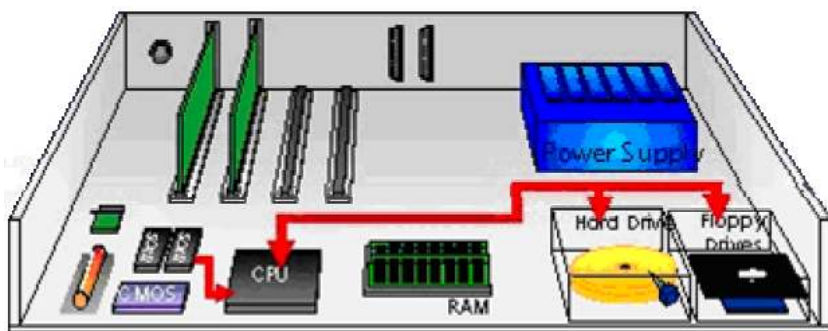


Figure 3.13: Conceptual legacy BIOS boot path. After power-on and firmware initialization, BIOS selects a boot device and can transfer control to code in its MBR. Modern UEFI systems follow a different path.

System and boot volumes. In Windows terminology, the *system* volume contains the files needed to start Windows, such as boot-manager files and boot configuration data. The *boot* volume contains the installed Windows operating-system files, including the kernel. Despite the names, the system volume starts the loading process and the boot volume contains the operating system being loaded; they can be the same partition or separate partitions [Microsoft, 2026b].

Modern UEFI booting. UEFI firmware initializes hardware, consults boot entries stored in nonvolatile firmware memory, and typically loads an EFI boot-manager program from the EFI System Partition (ESP). GPT is common with UEFI, but the partitioning scheme and firmware mode must be established from the evidence rather than assumed. The boot manager selects an operating system and loader; the loader initializes the kernel, which then starts drivers, services, and user-mode processes [UEFI Forum, 2024, Microsoft, 2026b].

Forensic questions may therefore include which firmware mode and boot order were configured, which disk and partition were selected, whether the MBR, volume boot record, EFI System Partition, boot configuration, or boot files changed, and whether an attached device or recovery environment could have affected the system state. Record such observations without changing firmware settings or booting an evidence device.

The boot-options view in Figure 3.14 emphasizes that firmware configuration can affect which device starts the system. Record such observations carefully and do not change boot settings merely to inspect them.

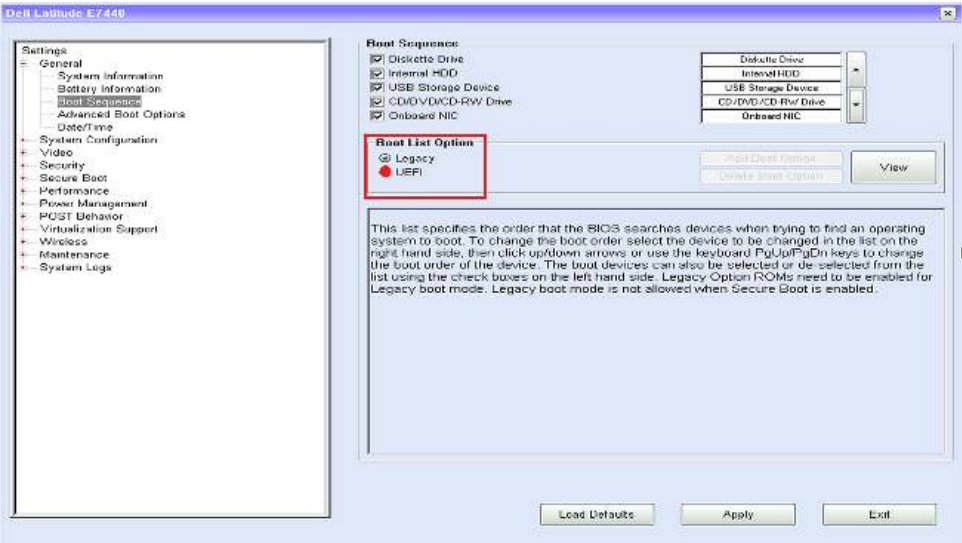


Figure 3.14: A firmware boot-options view. Boot configuration can provide useful context but should be inspected without changing the system state.

3.7 Master Boot Record

The Master Boot Record occupies the first logical sector of a legacy BIOS boot disk, traditionally a 512-byte sector. It contains bootstrap code, a partition table with four 16-byte entries, and the traditional 0x55AA boot signature. Figure 3.15 shows the conventional byte layout, including the disk signature and reserved bytes. An extended partition can link to additional logical partitions. During an examination, report the offset, raw bytes, and interpretation used to identify these structures [NeoSmart Technologies, 2026].

Illustrative MBR views. The following screenshots demonstrate how a hexadecimal viewer can expose MBR structures. They are included to help readers recognize the interfaces and the reasoning steps, not to prescribe a live-evidence

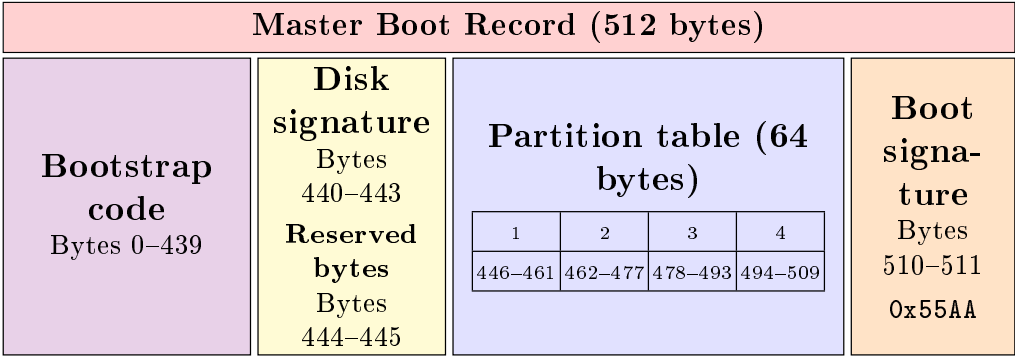


Figure 3.15: Conventional layout of a 512-byte Master Boot Record. The byte ranges identify the structures an examiner can locate in the first logical sector of a legacy BIOS boot disk.

workflow. Work on a verified copy whenever possible.

Tool choices. Both HxD and WinHex can view and edit hexadecimal data, including raw disk data. The screenshots in this section use WinHex, but the interpretation principles apply to either tool and to comparable hexadecimal viewers. Editing changes evidence: inspect a verified forensic copy or use a demonstrably read-only workflow, and record the tool, version, source, and mode used.

Figure 3.16 shows how to locate WinHex, and the red rectangle in Figure 3.17 identifies the **Open Disk** command used to begin opening a physical device.

After choosing **Open Disk**, WinHex displays the **Select Disk** dialog shown in Figure 3.18. The red rectangle highlights the physical storage device, not a logical volume or partition. In this laboratory example, HD0 is a VMware virtual disk presented to the virtual machine as a storage device. Before proceeding, compare the displayed size and identifier with the acquisition record.

Before examining the disk bytes, configure the hexadecimal display so that offsets and byte groups are easy to read. Figure 3.19 shows the **Options** → **General** path. In the General Options dialog, shown in Figure 3.20, enable hexadecimal offsets and use 16 bytes per line. These are display settings: they affect how bytes are presented, not the bytes stored on the disk.

Figure 3.21 shows how to display data-interpreter controls. Logical Block Ad-

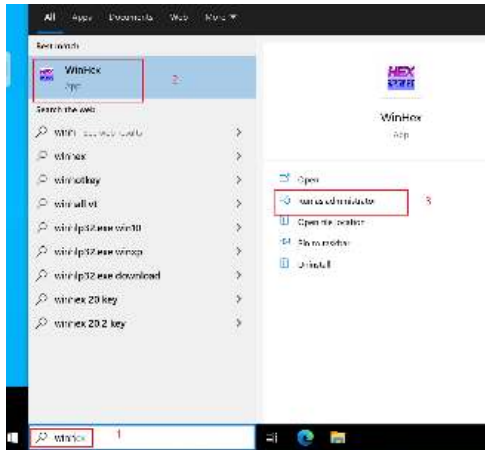


Figure 3.16: Launching WinHex.

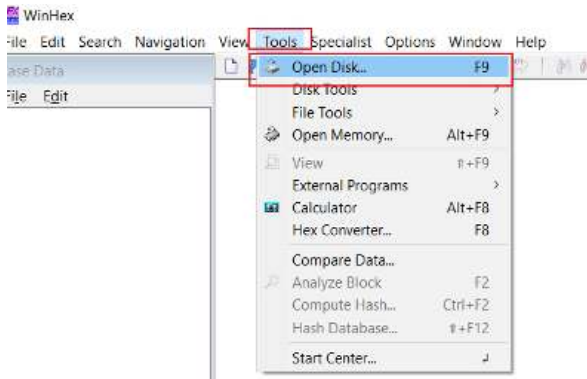


Figure 3.17: The **Open Disk** command in WinHex, highlighted in red. This is the menu step used to begin opening a physical device.

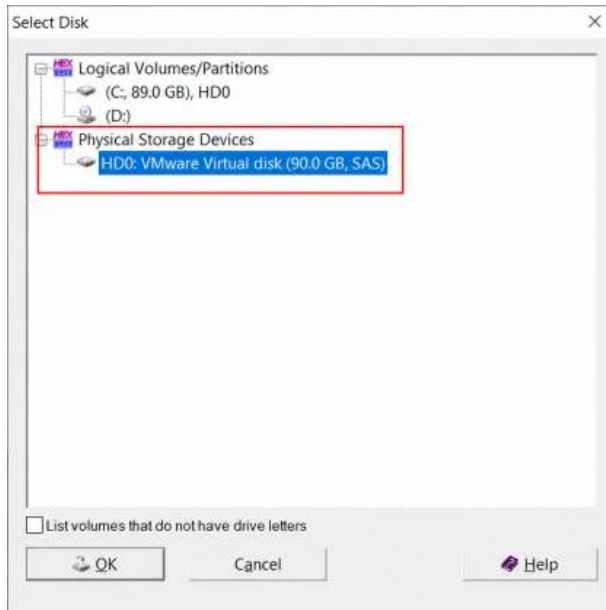


Figure 3.18: The **Select Disk** dialog displayed after choosing **Open Disk** in WinHex. The red rectangle identifies HD0, a VMware virtual disk presented to the laboratory virtual machine as a storage device; verify its identity before opening it.

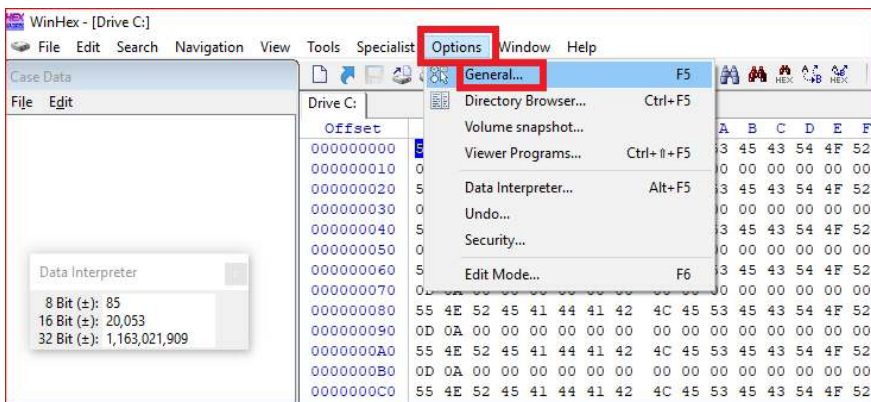


Figure 3.19: Opening WinHex General Options. The red rectangles identify the **Options** menu and the **General** command.

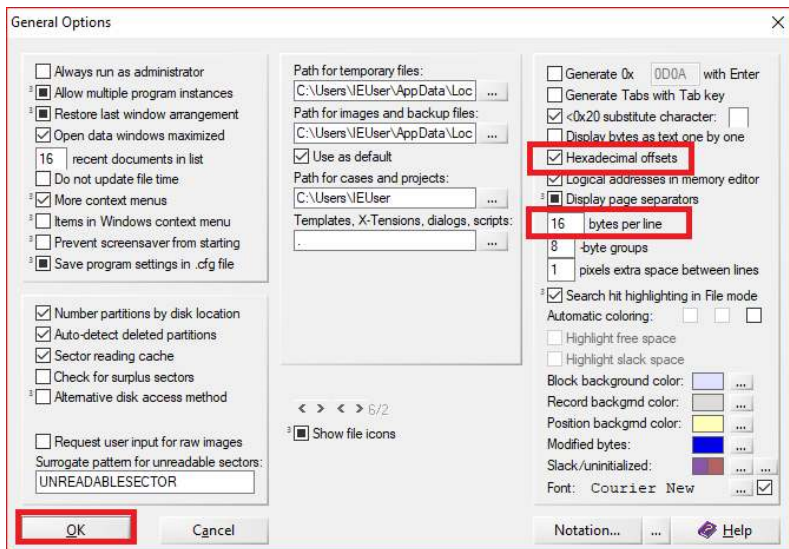


Figure 3.20: WinHex General Options for hexadecimal display. The red rectangles highlight hexadecimal offsets, 16 bytes per line, and **OK** to apply the display settings.

addressing (LBA) identifies each logical sector with a sequential number, starting at LBA 0. Multiply an LBA by the displayed bytes per sector to determine its byte offset. Figure 3.22 then displays the MBR in a hexadecimal view. Together, these views show why a byte sequence must be interpreted with an explicit representation, offset, and byte order.

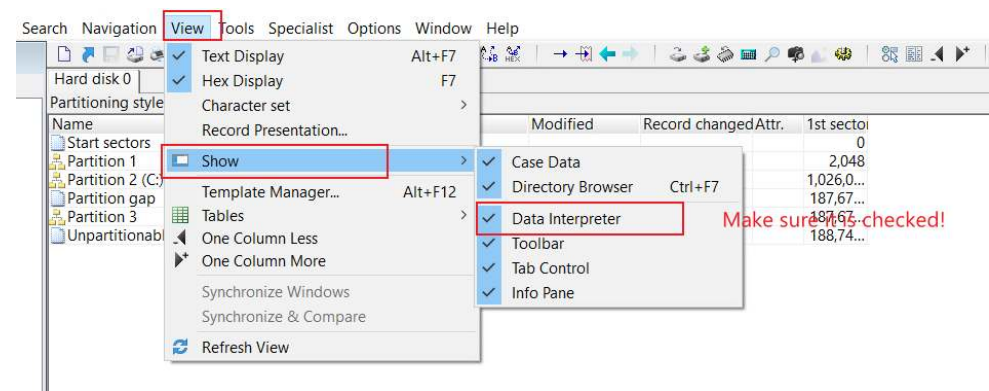


Figure 3.21: A hexadecimal viewer with data-interpreter controls highlighted. Record the selected interpretation when reporting a value derived from raw bytes.

Reading Partition 1's size. Figure 3.22 shows the MBR at LBA 0, not the first sector of the `C:` partition. An MBR partition entry contains both a starting LBA and a length in sectors. In the first 16-byte entry, the starting-LBA field is stored in little-endian order as `00 08 00 00`. Reversing the byte order gives `0x00000800`, or 2,048. With 512 bytes per sector, Partition 1 begins at byte offset:

$$2,048 \text{ sectors} \times 512 \text{ bytes/sector} = 1,048,576 \text{ bytes} = 1 \text{ MiB}.$$

The next four bytes, `00 F0 FF 04`, give the partition length. In little-endian order, this is `0x04FFF000`, or 83,881,984 sectors. In a Bash-compatible shell, `echo $((16*04fff000))` reproduces the hexadecimal-to-decimal conversion and outputs 83881984. Therefore:

$$\begin{aligned} 83,881,984 \text{ sectors} \times 512 \text{ bytes/sector} &= 42,947,575,808 \text{ bytes} \\ &= 39.998 \text{ GiB} \approx 40.0 \text{ GiB}. \end{aligned}$$

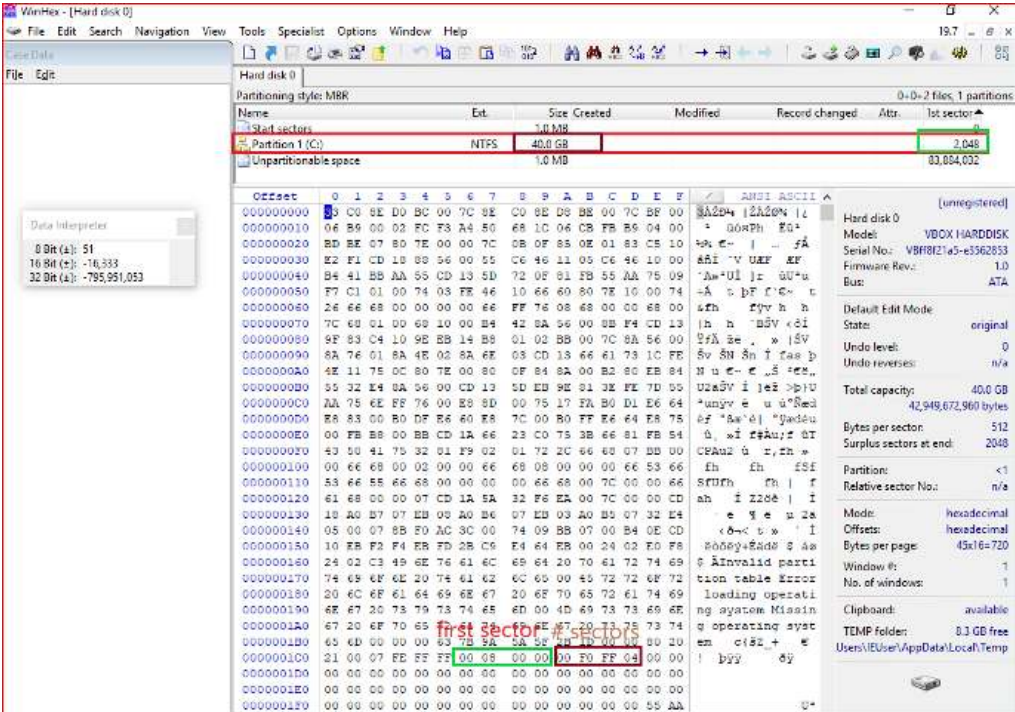


Figure 3.22: A hexadecimal view of the Master Boot Record (MBR), stored in the disk's first logical sector (LBA 0). The highlighted Partition 1 entry corresponds to the C: partition shown as approximately 40.0 GiB.

Sector numbers are logical block addresses (LBAs), normally counted from zero. If a tool reports a first sector of 1,024, it means LBA 1,024, which begins at byte offset $1,024 \times 512 = 524,288$ (512 KiB) on a 512-byte-sector disk. It is a sector address, not a number of “sections.” Always record the displayed sector size before converting an LBA to a byte offset.

Preserve the original bytes and document the calculation so another examiner can reproduce it.

3.8 GUID Partition Table

The GUID Partition Table (GPT) is a modern partitioning scheme that commonly accompanies UEFI booting. It was designed to overcome important limits of the older MBR format from the early PC era. MBR uses 32-bit logical block addresses (LBAs), which limit a disk to about 2 TiB when it uses 512-byte logical sectors. GPT uses 64-bit LBA fields, giving a theoretical addressable capacity of about 9.4 ZB (zettabytes); practical limits still depend on the firmware, operating system, storage device, and file system [UEFI Forum, 2026].

Modern storage tools use LBA rather than the older cylinder-head-sector (CHS) notation. An LBA identifies one logical sector in a sequential sequence, beginning with LBA 0. A logical sector is commonly 512 or 4,096 bytes, so an examiner must record the device’s reported logical-sector size before converting an LBA to a byte offset.

A GPT disk includes a protective MBR at LBA 0 for compatibility, a primary GPT header and partition-entry array near the beginning of the disk, and backup copies near the end. These redundant structures help an examiner validate partition boundaries and recover from limited corruption; they do not remove the need to corroborate fields against source bytes and an independent forensic tool.

The **protective MBR** is an MBR-format record at LBA 0 that helps protect a GPT disk from older MBR-only utilities. It typically contains a partition entry of type **0xEE**, indicating that the disk is already in use and should not be treated as an empty MBR disk. It is not the GPT partition table: the primary GPT header begins at LBA 1, followed by the GPT partition-entry array [UEFI Forum, 2024].

Begin with the high-level WinHex view in Figure 3.23. The red arrow and rectangle identify the **Partitioning style: GPT** label. The partition list identifies

the GPT layout, while the raw display is still at LBA 0 and therefore shows the protective MBR. The primary GPT header begins at LBA 1; do not mistake the LBA 0 bytes for that header.

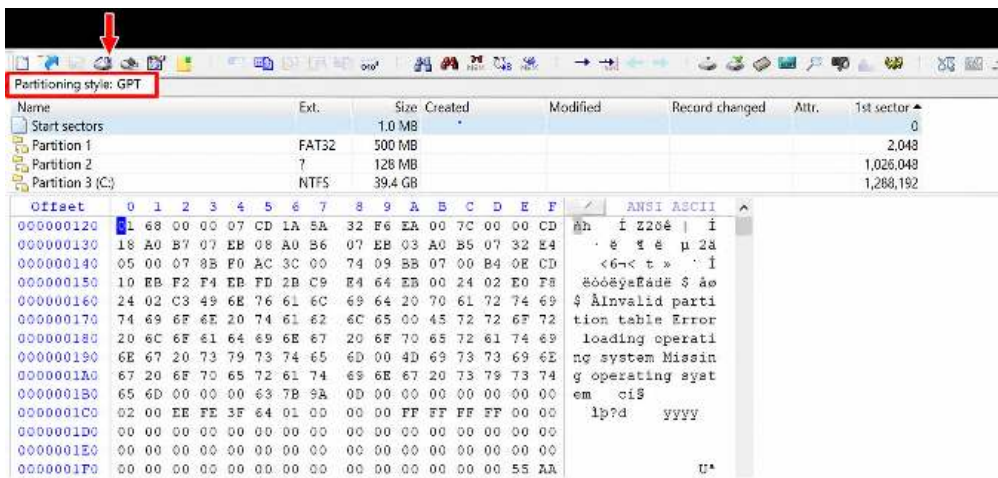


Figure 3.23: A GPT-partitioned laboratory disk in WinHex. The red annotation identifies the **Partitioning style: GPT** label; the partition list shows the GPT layout. The hexadecimal display at LBA 0 shows the protective MBR, and the primary GPT header begins at LBA 1.

Comparing the two laboratory disks. Compare the MBR disk in Figure 3.22 with the GPT disk in Figure 3.23. Table 3.3 summarizes their visible differences. Sizes retain the screenshots’ MB and GB labels; they are example values, not required partition sizes or independently verified decimal units.

The GPT layout is consistent with a Windows EFI System Partition (ESP), Microsoft Reserved partition (MSR), and Windows partition. Confirm the first two roles from their partition-type GUIDs in the GPT entries; size and file-system appearance alone are insufficient, and the screenshot does not establish the actual firmware boot mode. In a Windows UEFI installation, firmware loads Windows Boot Manager from the FAT32 ESP. The MSR instead reserves space for disk management and is not used to store ordinary user files or boot files. ESP and MSR sizes vary with deployment and Windows version; Microsoft’s current guidance specifies a 16 MB MSR, so the displayed 128 MB is not a universal requirement [Microsoft, 2026a,b].

Table 3.3: Observed layouts of the MBR disk in Figure 3.22 and the GPT disk in Figure 3.23. Suggested partition roles require confirmation.

Feature	MBR example	GPT example
Small FAT32 partition	None shown.	Partition 1: 500 MB; consistent with an ESP.
Possible MSR	None; MSR is a Windows GPT partition type.	Partition 2: 128 MB, file system shown as “?”; consistent with an MSR.
Main NTFS partition	Partition 1 (C:): 40.0 GB.	Partition 3 (C:): 39.4 GB.
Space outside partitions	1.0 MB “Start sectors” and 1.0 MB trailing “Unpartitionable space.”	1.0 MB “Start sectors” shown before Partition 1.

The MBR example shows only one NTFS partition, but a BIOS/MBR Windows installation can also have a separate NTFS *System Reserved* partition containing startup files. In the conventional BIOS/MBR boot path, MBR code transfers control to the active partition’s volume boot record; the system and Windows volumes may share a partition or occupy separate partitions [Microsoft, 2021, 2026b]. The trailing “Unpartitionable space” in this example lies outside the listed partition; its label alone does not explain why it was left unused. The leading “Start sectors” region includes disk metadata and should not be treated as entirely unused space. A partition without a drive letter may still contain boot-related evidence; identify its type and role before deciding what to examine.

Figures 3.24 and 3.25 then show the relationship among the GPT structures and provide a reference view of the fields an examiner should validate against the acquired disk image.

Reading the GPT layout. Read Figure 3.24 from top to bottom. LBA 0 holds the protective MBR and LBA 1 the primary GPT header. In the common 512-byte-sector configuration shown, LBAs 2–33 hold the primary partition-entry array (128 entries of 128 bytes each), and usable partition space begins at LBA 34. The figure therefore shows 128 entry slots; GPT records the actual entry count in its header and is not inherently limited to 128 partitions. Windows GPT disks, however, support up to 128 partitions [UEFI Forum, 2024, Microsoft, 2026a]. Near the end of the disk, GPT stores a backup entry array and backup GPT

header. Compare the primary and backup structures when validating boundaries and metadata; a mismatch is a finding to investigate and corroborate, not automatic proof of tampering.

GUID Partition Table Scheme

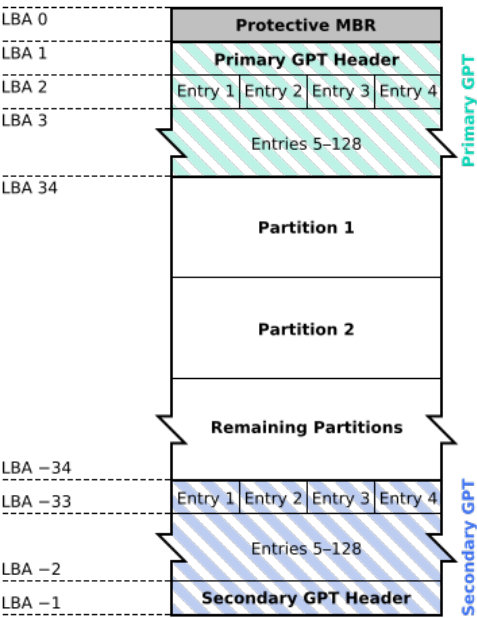


Figure 3.24: GUID Partition Table layout. The primary and backup GPT structures help an examiner corroborate partition boundaries and metadata.

Reading the GPT reference view. Figure 3.25 first provides a compatibility clue that the disk uses GPT. The MBR partition record begins at byte offset 0x1BE; its system-ID field at 0x1C2 contains EE, the GPT-protective type. Its starting-LBA field contains 01 00 00 00, which is little-endian for LBA 1. This protective-MBR entry is a useful indicator, but confirm GPT by examining the header at LBA 1 for the **EFI PART** signature and valid header fields.

On the 512-byte-sector disk shown, the GPT header begins at byte offset 0x200 (LBA 1), not 0x400. The **PartitionEntryLBA** field at header offset 0x48, or ab-

solute offset 0x248, contains 02 00 00 00 00 00 00 00. Interpreted as a little-endian 64-bit value, it is LBA 2. Therefore, $2 \times 512 = 1,024$ bytes, or 0x400, is the start of the partition-entry array shown in the figure. The header field determines this location; do not infer it from the offset alone [UEFI Forum, 2024].

Verifying the first partition’s size. Each GPT partition entry shown is 128 bytes (0x80), so the first entry occupies offsets 0x400 through 0x47F. Within that entry, the starting-LBA field begins at offset 0x20, or absolute offset 0x420. Its bytes are 22 00 00 00 00 00 00 00, which equal LBA 34 in little-endian order. The ending-LBA field begins eight bytes later, at 0x428; 21 00 01 00 00 00 00 00 equals 0x00000000000010021, or LBA 65,569. GPT ending LBAs are inclusive, so the partition occupies

$$65,569 - 34 + 1 = 65,536 \text{ sectors.}$$

With 512 bytes per sector, its size is

$$65,536 \times 512 = 33,554,432 \text{ bytes} = 32 \text{ MiB.}$$

This calculation verifies the first, Microsoft-reserved partition shown in the figure from the raw entry bytes rather than from its displayed label.



BY: JARED ATKINSON
TEMPLATE BY: ANGE ALBERTINI

PROTECTIVE MBR

FIRST SECTOR OF DRIVE
FOR BREAKDOWN SEE MOR FOSTER

180	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
181	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32
182	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33
183	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34
184	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35
185	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36
186	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37
187	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38
188	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39
189	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40
190	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41
191	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42
192	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43
193	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44
194	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	45
195	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46
196	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47
197	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47	

GPT HEADER

200	45	46	49	50	55	61	52	54	00	00	01	00	51	00	00	55
212	63	71	86	97					01	25	02	00	22	02	00	22
222	87	77	57	02	02	00	10	02	22	10	02	00	20	02	00	22
230	88	49	38	01	01	00	00	02	10	61	13	19	35	03	01	42
245	96	07	38	06	55	04	64	83	02	20	02	00	22	02	00	22
260	00	00	00	00	02	00	10	02	20	14	64	P8				

PARITION ARRAY

[illegible]

1. product
 2. test run
 3. reader used
 4. header CRC32
 5. my LBA
 6. alternate LBA
 7. test usable LBA
 8. test spare LBA
 9. disk gaid
 10. partition entry LBA
 11. # of partitions written
 12. # of partitions written
 13. partition entry array
 14. partition entry array CRC32

```

[7] PART
1.2
92
q797777]
1
22971919
M
22971486
[9] 2011-08-25 00:11:59.07-350:50:40:02
2 (vector containing of partition table)
120
128
00:00:00:00

```

partition type guid
unique partition guid
start ng | disk
ending LBA
off-boards
partition name

W7563 161010-4410-8174-50247001 Gen
H14547-0000-4346-4410-502470012121
34
G5509
d
Microsoft reserved direction

```
partition type guid
unique partition guid
start seq. no.
ending LBA
attributes
partition name
```

[illegible]

partition type guid
unique partition guid
starting LBA
size in bytes
partition name

```
0x0fa6c7-49e5-6475-87e0-6396e77209e7  
0277542a Ba4d-4f4d-9140 485812aaa027  
2164736  
4261f0f7  
U  
Enter data port ID
```

PARITION ARRAY

400	16	E3	C9	E3	5C	0B	B8	40	81	7D	F9	20	F0	02	15	AE
410	7F	8A	1A	FF	78	08	48	43	84	30	53	68	76	06	23	23
420	22	00	00	00	00	00	00	00	21	00	00	00	00	00	00	00
430	00	00	00	00	00	00	00	00	46	08	69	00	63	00	72	00
440	6F	00	73	00	66	06	66	74	20	20	20	72	00	65	00	00
450	73	00	65	00	72	76	05	00	65	64	00	20	00	70	70	00
460	61	00	72	00	74	69	00	74	69	69	00	6F	00	6E	00	00
470	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
480	A2	A0	D0	0B	E5	B9	33	44	87	C0	68	86	7B	26	99	C7
490	A2	AE	76	60	C3	9E	9F	4E	8D	43	30	3C	3F	60	36	76

```
partition type guid
unique partition guid
starting LBA
ending LBA
attributes
partition name
```

```
e3c9e316-0b5c-4db8-817d-f92df00215ae
ff1a8a47-08f8-43ab-b410-53697f0b2323
34
65569
0
Microsoft reserved partition
```

Figure 3.25: GPT reference view and close-up of its first partition entry. The full view establishes the protective MBR, GPT header, and partition-entry array. In the close-up, the red outline identifies the 16 bytes that contain the starting LBA (22 00 00 00 00 00 00 00) and ending LBA (21 00 01 00 00 00 00 00) used to verify the 32 MiB partition size. *Graphic credit: Jared Atkinson (Invoke-IR), marked CC BY; template credit: Ange Albertini.*

3.9 Summary

Digital evidence can originate at every layer of a computer system: user activity and applications, the operating system, boot components, and physical hardware. Processors and memory provide volatile context, while storage devices, partitions, partition tables, network interfaces, and peripherals can preserve longer-lived artifacts. Examiners need to understand this system context, document it carefully, and correlate multiple sources before drawing conclusions.

Chapter Review

Review Questions

1. Name three computer-system layers and give one potential evidence source from each.
2. Why can a memory-collection decision differ from a storage-image acquisition decision?
3. What is the difference between a partition and a volume?
4. How can an MBR or GPT help an examiner understand a storage device and its boot context?
5. Why might the absence of an expected artifact not prove that an action did not occur?

Discussion Questions

1. A laptop has an external drive attached when an incident is reported. What facts should be documented before deciding whether to acquire the drive, and why?
2. How could a cloud-synchronization client or a software update complicate an interpretation of local file timestamps?

Hands-On Exercise

Exercise. Using Figures 3.13, 3.15, and 3.24, create a short evidence note for a hypothetical computer. State which boot path is consistent with a legacy

BIOS/MBR system and which is consistent with a UEFI/GPT system. For each path, identify two artifacts an examiner should preserve or document and explain why booting the evidence device would be inappropriate.

4 Examining Digital Evidence with Linux

Learning Objectives

- Explain the Linux filesystem hierarchy, virtual file system, and inode concepts.
- Navigate directories and inspect files with core Linux commands.
- Search files and directories for evidence-relevant content.
- Interpret basic network and process information from Linux utilities.
- Create, execute, and permission a simple shell script safely.

In digital forensics, the choice of operating system is pivotal to the success of an investigation. The operating system (OS), along with the file system, plays a pivotal role in digital forensics investigations. Understanding their intricacies is crucial for successfully acquiring, analyzing, and interpreting digital evidence. Out of all the available OS, Linux is favored by many professionals for its open-source architecture, which offers unmatched flexibility and control. Its stability and security make it ideal for forensic analysis, allowing investigators to explore digital evidence without the constraints found in other operating systems.

Beyond its robust framework, Linux provides an extensive array of specialized tools that are crucial for tasks like data recovery and network analysis. These tools, designed specifically for forensic analysis, empower investigators to conduct thorough and precise investigations. This chapter will explore why Linux is indispensable in digital forensics and introduce key tools and techniques used by experts in the field.

4.1 Introduction to Operating System (OS)

An **Operating System (OS)** is the foundational software that manages a computer's hardware and software resources, enabling various programs to run efficiently. It acts as an intermediary between users and the computer hardware, ensuring that different applications can operate concurrently without conflict. The OS handles critical tasks such as memory management, process scheduling, file management, and device control, allowing users to interact with the computer system in a seamless and user-friendly manner. Popular operating systems include Windows, macOS, Linux, and Android, each catering to different types of devices and user needs.

In essence, the OS is the backbone of a computer system, coordinating all activities and providing the necessary environment for applications to function. It simplifies complex operations, making computing accessible and efficient for users, whether they are navigating a desktop interface, running applications, or managing files and hardware.

Figure 4.1 places these operating systems within a broader OS family tree.

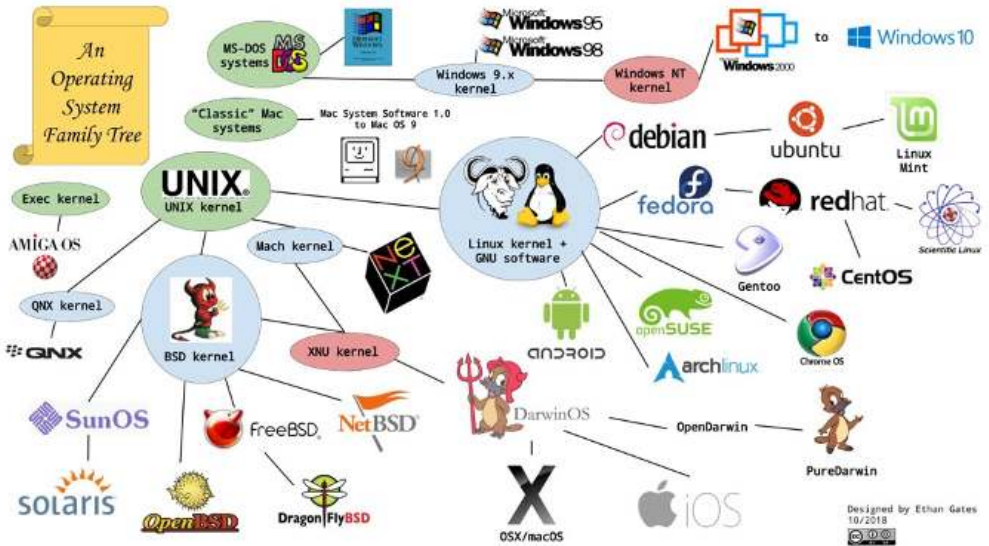


Figure 4.1: OS and Kernel Family Tree. Source: EG-tech, digipres-posters; licensed CC BY-SA 4.0.

Examples of Operating Systems.

Microsoft Windows The most widely used OS for personal computers, known for its graphical user interface and broad software compatibility.

macOS The operating system used by Apple computers, recognized for its user-friendly interface and integration with Apple's ecosystem.

Linux An open-source OS available in various distributions such as Ubuntu, Fedora, and Debian, favored for its flexibility, customization options, and stability.

Android A Linux-based mobile OS predominantly used in smartphones and tablets, offering a vast app ecosystem.

iOS The mobile OS powering Apple's iPhones and iPads, known for its smooth performance and intuitive interface.

Key functions of an Operating System. An Operating System acts as the communication layer between computer hardware and the user. It coordinates hardware access, provides a working environment for software, and manages how

data moves through the system. Its main responsibilities include:

Hardware Management The OS oversees the allocation and management of hardware resources. It determines which programs get access to the CPU, how much memory is allocated to each process, and controls the flow of data to and from storage devices and peripherals.

Software Management The OS provides a runtime environment for application software. It loads programs into memory, schedules their execution on the CPU, facilitates communication between different programs, and handles any errors or exceptions that may occur.

User Interface The OS offers a user interface, which can be a graphical user interface (GUI) with windows, icons, and menus, or a command-line interface (CLI) where users interact with the system through text commands. The user interface allows users to launch applications, manage files, and configure system settings.

File Management The OS organizes and manages files and directories on storage devices through a file system. It handles file creation, deletion, modification, and access control, ensuring data integrity and security.

Security The OS implements security features to protect the system and data from unauthorized access, malware, and other threats. These features include user authentication, access control mechanisms, and firewalls.

Networking Many modern operating systems provide built-in networking capabilities, enabling computers to connect to networks and communicate with other devices. This includes support for protocols like TCP/IP, which form the foundation of the Internet.

Device Drivers The OS manages device drivers, which are software modules that allow the OS to communicate with and control various hardware devices connected to the computer.

4.2 Introduction to File System

A **file system** is a crucial component of an operating system that defines how data is organized, stored, and accessed on storage devices like hard drives, SSDs, or USB flash drives. It provides a structured way to manage and retrieve files, enabling users to interact with their data effectively.

Figure 4.2 shows a simplified layout of how a file system organizes files and

directories.

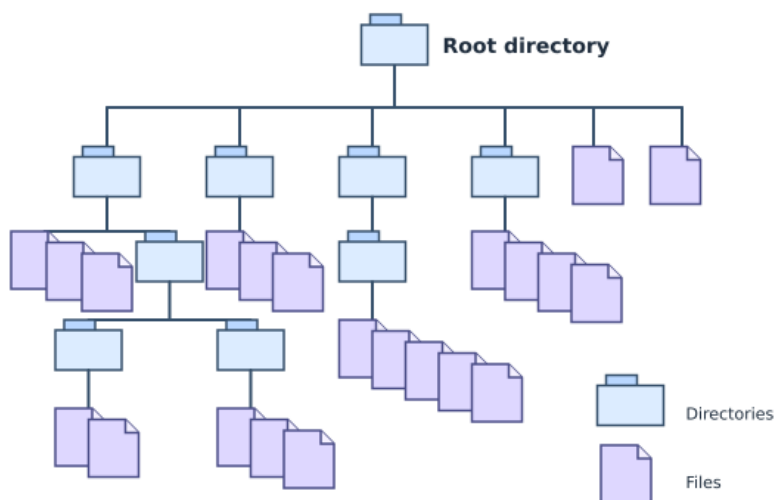


Figure 4.2: Basic layout of file system

Files At its core, a file is a container for storing related information. This information could be anything from text documents and images to executable programs and system configuration data. Files are typically stored on secondary or non-volatile storage media, such as hard drives or solid-state drives, ensuring that the data persists even when the computer is turned off.

File Systems A file system acts as the underlying framework for managing files on a storage device. It defines the rules and structures that govern how files are named, stored, and retrieved.

Named The file system determines the naming conventions for files, including the allowed characters, maximum length, and case sensitivity.

Stored The file system organizes files into directories (or folders) and allocates storage space on the disk for each file. It keeps track of where each file is located on the storage device.

Retrieved The file system provides mechanisms for locating and accessing files based on their names and directory paths.

File systems used by operating systems. Investigators need to be familiar

with the structure and characteristics of various file systems to effectively acquire and analyze digital evidence. They use specialized tools and techniques to examine file system metadata, recover deleted files, and reconstruct file system activities to uncover potential evidence of crimes or malicious activities. Different OS utilize various file systems, each with its own set of features and advantages:

Linux Common Linux file systems include **ext4**, a widely used format known for performance and stability; **XFS**, a scalable journaling file system suited to large files and volumes; **JFS**, which is lightweight and efficient; and **btrfs**, which adds modern features such as snapshots, checksums, and transparent compression.

Solaris **ZFS** is a highly scalable and reliable file system with built-in features such as data-integrity checks, snapshots, and compression.

macOS **APFS** is the default file system for modern macOS systems and is optimized for solid-state drives, while **HFS+** remains relevant mainly for older systems and legacy compatibility.

Microsoft Windows **FAT** is still used for compatibility with older systems and removable storage, while **NTFS** is the standard Windows file system and offers stronger security, journaling, and support for larger files and volumes.

4.3 The Virtual File System: A universal translator for file systems

In the intricate world of operating systems, where diverse file systems coexist, the **Virtual File System** (VFS) emerges as a unifying force, providing a common language for interacting with files and directories across different storage platforms. It acts as a bridge between user-level processes and the underlying file systems, abstracting their complexities and offering a consistent interface for file operations.

At its core, the **VFS** is an abstraction layer within the operating system's kernel. It creates a virtualized representation of the file system hierarchy, shielding applications from the intricacies of specific file system implementations. This abstraction allows programs to work seamlessly with files, regardless of whether they reside on a local hard drive formatted with NTFS, a network-attached storage device using NFS, or even within a compressed archive like a ZIP file.

The VFS comprises several key components that work in concert to provide its functionality:

- **Vnode (Virtual Node):** The fundamental building block of the VFS, a vnode represents a file or directory within the virtual file system hierarchy. It stores essential information about the file, such as its type, size, permissions, and location on the underlying file system.
- **File Operations:** A collection of functions that define how to interact with files. Each file system implements its own set of file operations, which are registered with the VFS.
- **Superblock:** Contains metadata about a specific file system, such as its type, block size, and other configuration parameters.
- **Mount Point:** The designated location within the file system hierarchy where a particular file system is attached, making its contents accessible through the VFS.

For example, a Linux system might support ext4, NTFS, and FAT32 file systems simultaneously. Thanks to VFS, applications can interact with files stored on these diverse file systems using the same set of commands and interfaces, such as **open**, **read**, **write**, and **close**. The VFS handles translating these commands into the appropriate actions for the specific file system in use, providing a seamless experience for both the user and the application.

Unified representation of devices. One of the most powerful aspects of the Linux VFS is its ability to represent not just regular files and directories, but also hardware devices as file-like objects. In Linux, everything from hard disks and CD/DVD drives to network interface cards (NICs) and USB devices is treated as a file. These devices are typically represented within the `/dev/` directory, where each device is given a corresponding file that users and applications can interact with.

For example, a hard disk might be represented by the file `/dev/sda`, and a USB drive might appear as `/dev/sdb1`. By treating devices as files, Linux allows users to perform standard file operations—such as reading, writing, and executing—on hardware components. This uniform approach simplifies the management and investigation of devices, making it easier to apply the same techniques and tools to different types of hardware.

Figure 4.3 summarizes this relationship as a layered model. User programs issue

familiar file operations at the top. The VFS sits in the middle as the abstraction layer, presenting one consistent interface even when the data underneath lives on very different file systems. Below that, Linux can route requests to local file systems such as **ext4** or **XFS**, foreign file systems such as **NTFS** or **FAT/exFAT**, optical-media formats such as **ISO 9660**, or network file systems such as **NFS** and **SMB**. The block layer and device drivers then translate those requests into operations against the actual storage hardware.

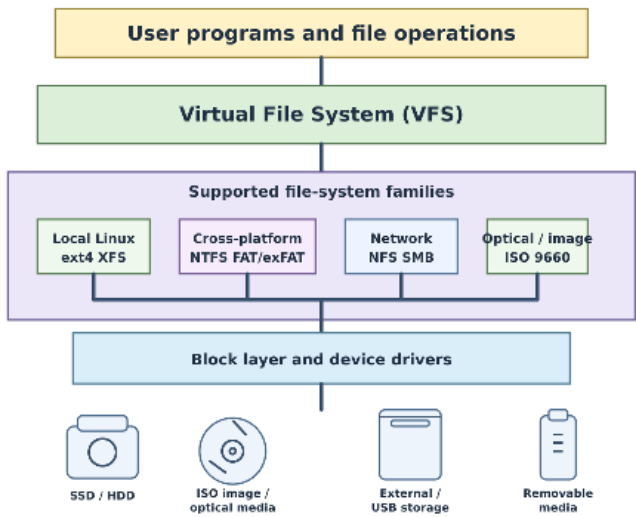


Figure 4.3: Layered view of the Linux Virtual File System

The key point to notice in Figure 4.3 is that the investigator does not need a different mental model for every storage technology. Whether the evidence comes from an internal Linux volume, a Windows-formatted removable drive, a mounted ISO image, or a network share, the VFS presents a common entry point for file operations while the lower layers handle the storage-specific details.

In summary, Linux’s Virtual File System (VFS) is a crucial component that provides a uniform interface for accessing and managing different file systems and devices. By abstracting the complexities of individual file systems and treating everything as a file, Linux ensures that applications and users can interact with the system consistently and predictably. This approach not only enhances the

usability and flexibility of the operating system but also supports a wide range of applications, from everyday computing to specialized fields like digital forensics.

4.4 Navigating the Linux command line

Having established the pivotal role of Linux in the field of digital forensics, it's imperative to equip investigators with the fundamental skills required to navigate its command-line interface. This section serves as a practical guide to essential Linux operations, focusing on those directly applicable to forensic investigations. From launching the terminal and understanding the file system structure to viewing file details and utilizing powerful commands, we'll lay the groundwork for effectively harnessing the capabilities of Linux in the pursuit of digital evidence.

4.4.1 Getting started in the terminal

Launching the Linux Terminal. In the Kali Linux distribution, there are multiple terminal emulators available. Let's focus on accessing the "Terminator" terminal, as highlighted in Figure 4.4.

Locate the Application Launcher Click the Kali dragon icon in the bottom-left corner to open the application launcher.

Search for "Terminator" In the search bar, type "ter" to filter the list of applications. The search results display "Terminator" as one of the options.

Launch Terminator Click the "Terminator" entry in the search results to open the terminal emulator.

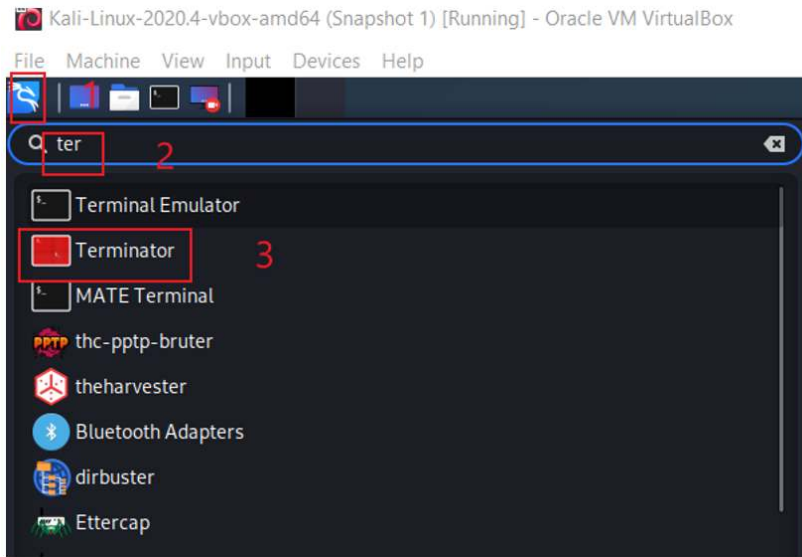


Figure 4.4: Accessing the Terminator terminal emulator in Kali Linux

Keeping your system up-to-date. In digital forensics, ensuring the integrity and security of your investigative environment is paramount. One crucial aspect of this is keeping your Linux system and its software packages up-to-date. This practice not only provides access to the latest features and bug fixes but also safeguards your system from potential vulnerabilities that could be exploited during investigations.

The **apt-get update** command serves as the first step in this maintenance process. It refreshes the local package lists on your system, ensuring that you have the most current information about available software packages from various repositories. These repositories act as centralized storage locations for software packages, allowing for easy installation and updates.

Figure 4.5 demonstrates the output of the **apt-get update** command. Each line details the retrieval of a package list from a specific repository. For instance, `Get:1 http://... InRelease [3,232 B]` indicates that the package list for a Microsoft repository was successfully downloaded, and its size is 3,232 bytes.

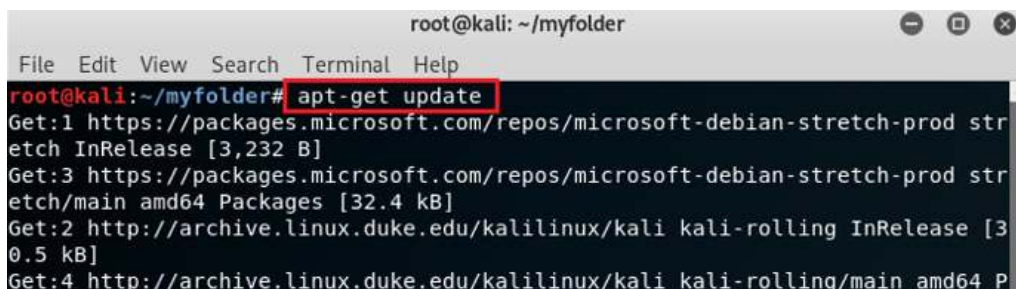
A terminal window titled 'root@kali: ~/myfolder' with a menu bar (File, Edit, View, Search, Terminal, Help). The command 'apt-get update' is entered and highlighted with a red box. The output shows four lines of progress: 'Get:1 https://packages.microsoft.com/repos/microsoft-debian-stretch-prod stretch InRelease [3,232 B]', 'Get:3 https://packages.microsoft.com/repos/microsoft-debian-stretch-prod stretch/main amd64 Packages [32.4 kB]', 'Get:2 http://archive.linux.duke.edu/kalilinux/kali kali-rolling InRelease [30.5 kB]', and 'Get:4 http://archive.linux.duke.edu/kalilinux/kali kali-rolling/main amd64 P'.

Figure 4.5: Output showing the process of updating package lists from various repositories

Regularly updating package lists using `apt-get update` is a foundational practice in maintaining a secure and reliable Linux system for forensic investigations. This ensures that you have access to the latest software versions and security patches, mitigating potential risks and enabling you to conduct investigations with confidence.

Installing Terminator in Linux command line. In the realm of digital forensics, maintaining an up-to-date and well-equipped investigative environment is crucial. Linux’s package management system, `apt-get` is a powerful command-line tool used for managing software packages on Linux systems. It works with the **Advanced Package Tool (APT)** system to handle package installation, removal, updating, and other package-related tasks.

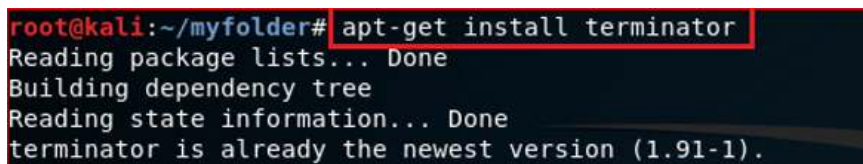
A terminal window showing the command 'apt-get install terminator' highlighted with a red box. The output consists of four lines: 'Reading package lists... Done', 'Building dependency tree', 'Reading state information... Done', and 'terminator is already the newest version (1.91-1)'.

Figure 4.6: Linux command to install terminator.

Figure 4.6 showcases an attempt to install the “Terminator” terminal emulator using the command `apt-get install terminator`. While the output indicates that Terminator is already installed and up-to-date on this system, it still provides a useful example of command structure:

apt-get The Linux package-management command used to install, remove, and update software from configured repositories.

install Tells **apt-get** that the next argument names a package to be installed.

terminator The package name of the terminal emulator being requested.

What to notice in the output. The system reports that Terminator is already the newest version. That tells the reader the command succeeded, but no additional installation was needed.

Customizing the Terminal Layout in Terminator. Terminator, as highlighted in the previous section, is a powerful terminal emulator known for its ability to split terminal windows into multiple panes. This feature significantly enhances productivity by allowing simultaneous interaction with multiple command-line sessions within a single window.

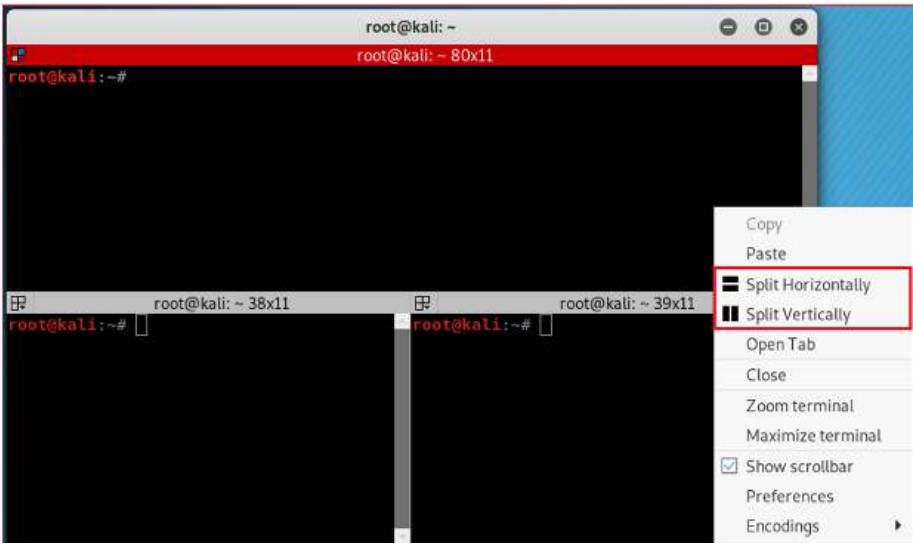


Figure 4.7: Layout of the terminator.

Figure 4.7 provides a visual representation of Terminator’s layout capabilities.

Split Panes The terminal window is divided into two vertical panes, each displaying a separate shell session. The dimensions shown in the screenshot indicate that each pane is an independent workspace.

Split Horizontally / Split Vertically These context-menu options let the inves-

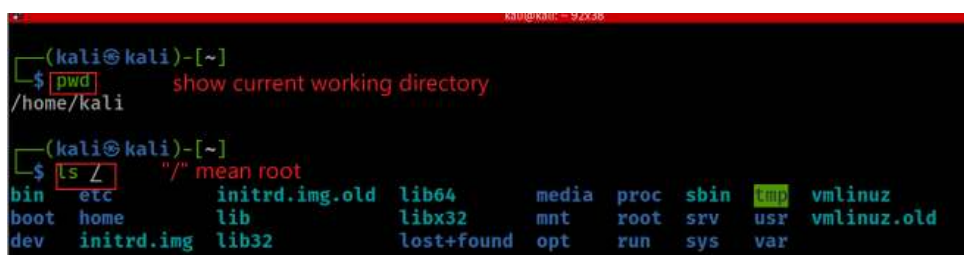
tigator divide the current pane into additional work areas.

Open Tab, Close, Zoom, Maximize These controls manage either the selected terminal session or the overall window layout.

Show Scrollbar / Preferences These options adjust the terminal's appearance and behavior.

Encodings This option manages the character-encoding settings used by the terminal.

Viewing the Linux file system from root. Now let's explore how to view the contents of the root directory, which serves as the starting point for the entire file system hierarchy. In Linux, "**root directory**" is symbolized by the **"/ (forward slash)"** and the "**home directory**" is symbolized using the **"~(tilde)"**. Thus to view the content of the root directory **ls /** command is used. Figure 4.8 shows the use of this command.



```
(kali㉿kali)-[~]  
$ pwd  
/home/kali  
  
(kali㉿kali)-[~]  
$ ls /  
bin      etc      initrd.img.old  lib64      media  proc  sbin  tmp  vmlinuz  
boot    home     lib             libx32     mnt    root  srv   usr  vmlinuz.old  
dev     initrd.img  lib32          lost+found  opt    run   sys   var
```

Figure 4.8: Understanding the Linux file system hierarchy and the contents of the root directory

pwd Stands for “print working directory” and shows the directory the user is currently in. In this example, the output is **/home/kali**.

ls / Lists the contents of the **root directory (/)** rather than the current directory.

What to notice in the output. The listing includes standard top-level directories such as **bin, boot, dev, etc, home, lib, tmp, usr, and var**. For a forensic examiner, this confirms that **/** is the top of the Linux filesystem hierarchy and that every other directory branches from it.

Figure 4.8 demonstrates basic navigation and file system exploration within a Kali Linux environment, showcasing how to view the contents of the root directory using the command line.

Clearing Linux terminator. As we continue navigating the Linux command line, decluttering the terminal screen is often useful for better readability and focus. The `clear` command serves precisely this purpose.



Figure 4.9: The `clear` command being executed in a Kali Linux terminal to clear the screen

The `clear` command removes the visible text from the terminal window, giving the investigator a clean screen before the next task. It does not erase command history, so previously entered commands can still be recalled with the up-arrow key. Figure 4.9 shows the command being issued from the terminal prompt before the session continues.

4.4.2 Exploring files and directories from the terminal

Viewing detailed File and directory information. In the previous sections, we’ve explored how to list the contents of directories using the `ls` command. However, in digital forensic investigations, we often need more than just a simple list of filenames. Understanding file permissions, ownership, modification dates, and other metadata is crucial for assessing files’ relevance and potential evidentiary value.

The `ls -l` command, demonstrated in Figure 4.10 provides a detailed or “long listing” format that reveals a wealth of information about each file and directory. Here’s how it does:

ls The basic command used to list directory contents.

-l Requests the long-listing format so additional metadata is shown for each entry.

```

(kali@kali)-[~]
$ ls -l /
total 68
lrwxrwxrwx 1 root root 7 Nov 17 2020 bin -> usr/bin
drwxr-xr-x 3 root root 4096 Nov 17 2020 boot
drwxr-xr-x 17 root root 3360 Jun 27 15:41 dev
drwxr-xr-x 157 root root 12288 Jun 27 15:29 etc
drwxr-xr-x 3 root root 4096 Nov 17 2020 home
lrwxrwxrwx 1 root root 33 Nov 17 2020 initrd.img -> boot/initrd.img-5.9.0-kali1-amd64
lrwxrwxrwx 1 root root 33 Nov 17 2020 initrd.img.old -> boot/initrd.img-5.9.0-kali1-amd64
lrwxrwxrwx 1 root root 7 Nov 17 2020 lib -> usr/lib
lrwxrwxrwx 1 root root 9 Nov 17 2020 lib32 -> usr/lib32
lrwxrwxrwx 1 root root 9 Nov 17 2020 lib64 -> usr/lib64
lrwxrwxrwx 1 root root 10 Nov 17 2020 libx32 -> usr/libx32
drwx----- 2 root root 16384 Nov 17 2020 lost+found

```

Figure 4.10: Detailed listing of the root directory contents

The output (Figure 4.10) of command `ls -l` presents several columns that matter during interpretation:

Permissions The first column shows the permission string, such as `-rwxr-xr-x`.

Hard Link Count Indicates how many hard links point to the file.

Owner Displays the username of the file owner.

Group Shows the group associated with the file.

Size Reports the file size in bytes.

Date & Time Shows the last modification time.

Name Displays the file or directory name.

Visualizing the Linux file system with the tree command. In addition to the `ls` command, which provides a linear list of files and directories, Linux offers the `tree` command to visualize the file system's hierarchical structure. This can be particularly useful in digital forensics investigations when understanding the relationships between files and directories is crucial.

The command `tree -L 1 /`, as demonstrated in Figure 4.11, displays a one-level tree structure of the root directory `/`. Let's break it down:

tree Displays a hierarchical view of directories and files.

-L 1 Limits the depth to one level so only the immediate contents of the target directory are shown.

/ Specifies the root directory as the starting point.

```
(kali㉿kali)-[~]  
$ tree -L 1 / only show Level 1  
├── bin -> usr/bin  
├── boot  
├── dev  
├── etc  
├── home  
├── initrd.img -> boot/initrd.img-5.9.0-kali1-amd64  
├── initrd.img.old -> boot/initrd.img-5.9.0-kali1-amd64  
├── lib -> usr/lib  
├── lib32 -> usr/lib32  
├── lib64 -> usr/lib64  
├── libx32 -> usr/libx32  
├── lost+found  
├── media  
└── mnt
```

Figure 4.11: One-level tree view of the root directory

What to notice in the output.

Root at the top The tree starts from /, confirming that the command is showing the top of the filesystem hierarchy.

Directory markers Directories appear with a trailing slash.

Parent-child structure Files and directories are visually nested under their parent locations.

Symbolic links Links are marked with ->, followed by their target.

Note

The **tree -help** command provides detailed information about the various options available to customize the output of the tree command, allowing for further filtering and exploration.

Displaying current working directory contents. The **pwd** command in Linux stands for “**print working directory**”. When executed, it outputs the absolute path of the current working directory. While the **pwd** command directly reveals your current working directory, the **tree** command offers a more visual and structured representation of its contents, particularly when combined with the **-L** option to control the depth of the displayed hierarchy.